2D Graph Visualization

Node-Link Diagrams · Adjacency Matrices · Layout Algorithms · Hybrid Systems

Raz Saremi, Ph.D.

Tandon School of Engineering, NYU

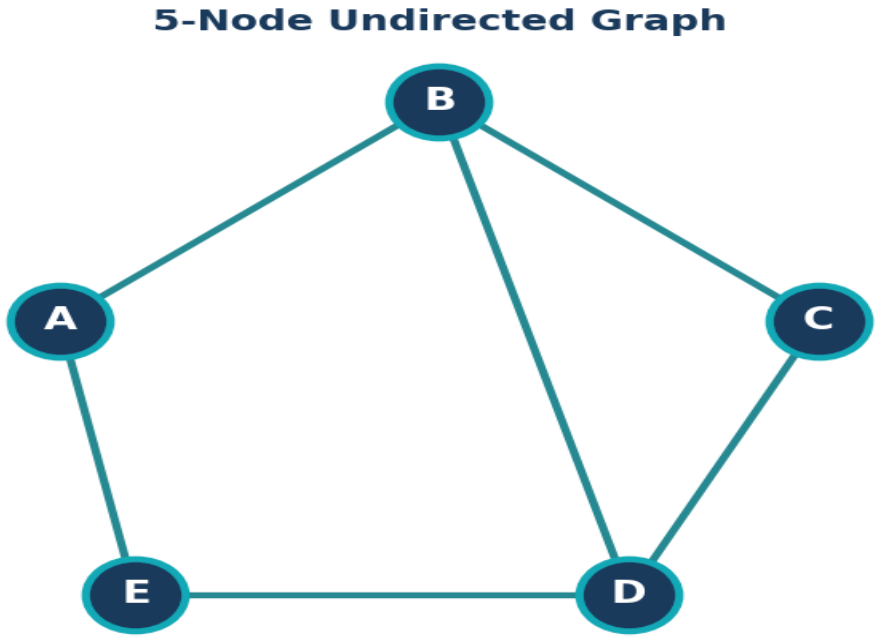
Information Visualization

What is a Graph? G = (V, E)

Formal definition with real-world examples.

Formal Definition G = (V, E)

V	Set of Vertices (Nodes)
E	Set of Edges (Links)
|V|	Number of nodes
|E|	Number of edges



Real-World Examples

- Social networks (people → friends)
- The Internet (servers → routers)
- Software systems (modules → deps)
- Biological networks (genes → proteins)

The Challenge of Scale: Small vs. Large Networks

Max Possible Edges = n(n−1) / 2

Nodes	Max Possible Edges
10	45
50	1,225
100	4,950
500	124,750
1,000	499,500

Small Networks (< 100 nodes)

- Density < 0.3 typically
- Node-Link works well
- Interactive exploration feasible

Large Networks (> 100 nodes)

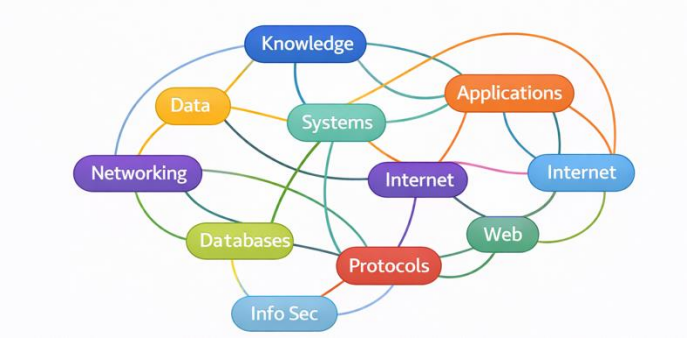
- 100k+ edges possible
- "Hairball" effect emerges
- Adjacency Matrix scales better

Key Insight: The right visualization depends on network size AND density.

Visualization Goals: Readability, Pathfinding & Cluster Detection

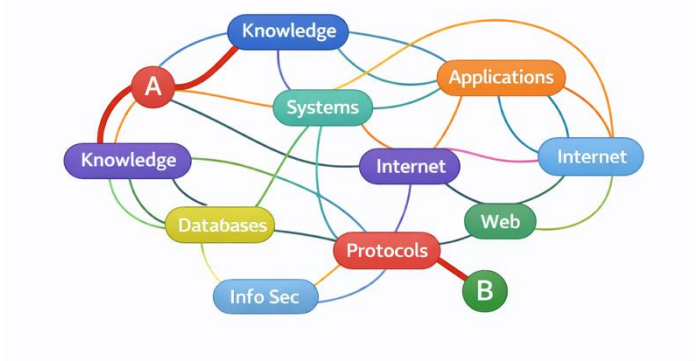
Readability

Minimize edge crossings and node overlaps. Text labels must remain legible at target zoom level.



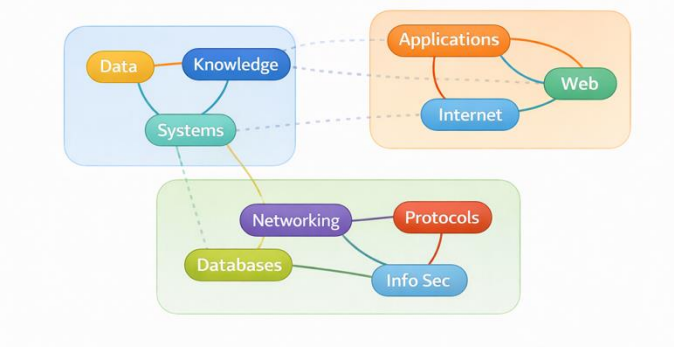
Path-finding

Can a user trace a route from A to B? Critical for network navigation and traversal tasks.



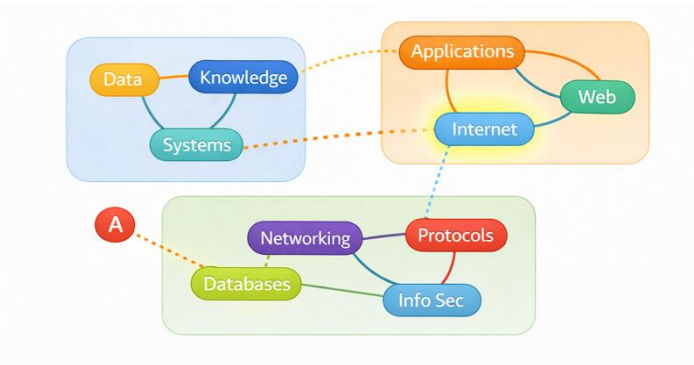
Cluster Detection

Densely connected node groups should be visually separable from other communities.



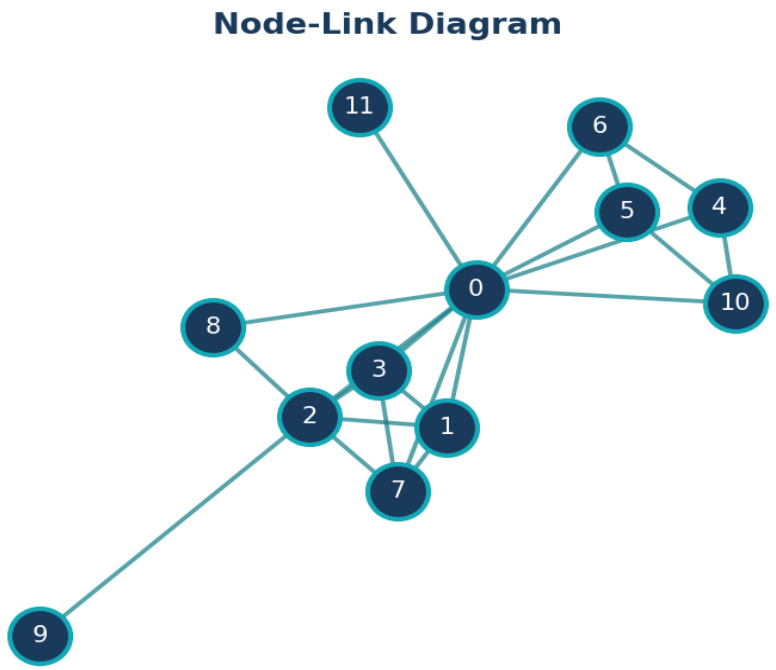
Outlier Spotting

Isolated nodes and bridge edges that connect separate components must visually stand out.



Overview: The Two Main Paradigms

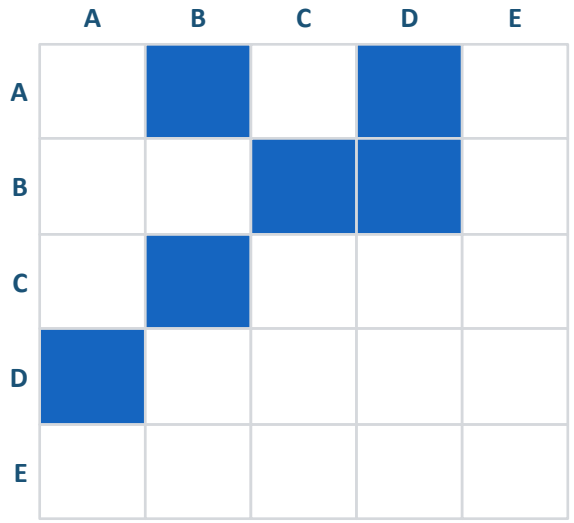
Every graph visualization uses one of these two foundational approaches.



Node-Link Diagram (NL)

Nodes as circles, edges as lines. Intuitive & familiar metaphor. Best for sparse graphs below ~200 nodes.

Adjacency Matrix (AM)

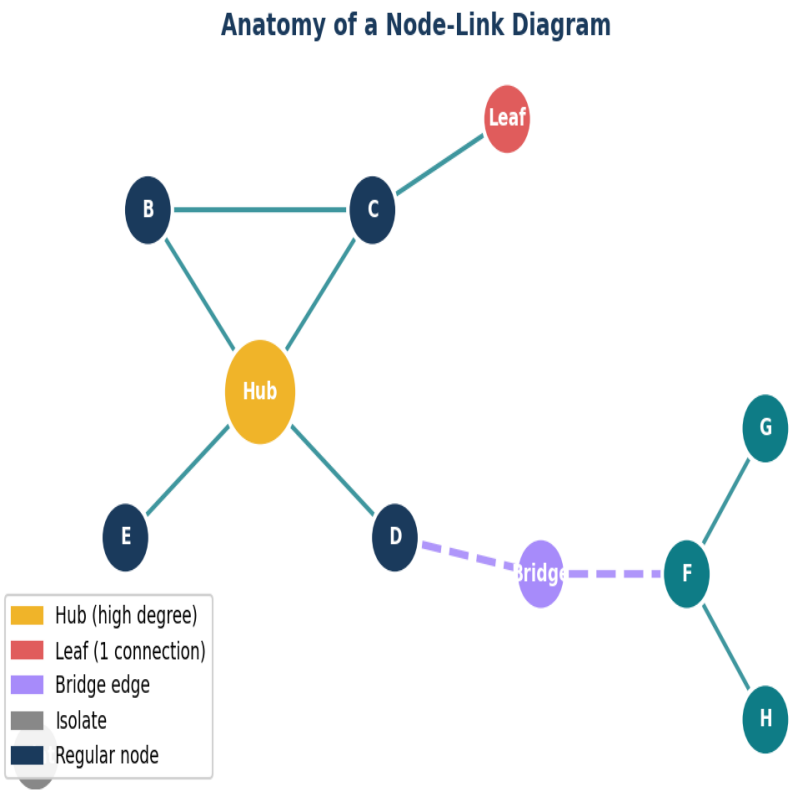


No edge crossings — every edge is a filled cell. Ideal for dense/large graphs.

Choosing between NL and AM is the core decision in graph visualization. It depends on size, density, and the user's task.

Anatomy of a Node-Link Diagram

Understanding the visual vocabulary of node-link graphs.



Hub / High-degree node

Many connections. Visually central in force-directed layouts.

Leaf node

Only one connection. Found at graph periphery.

Bridge edge

Single edge whose removal disconnects the graph.

Isolate

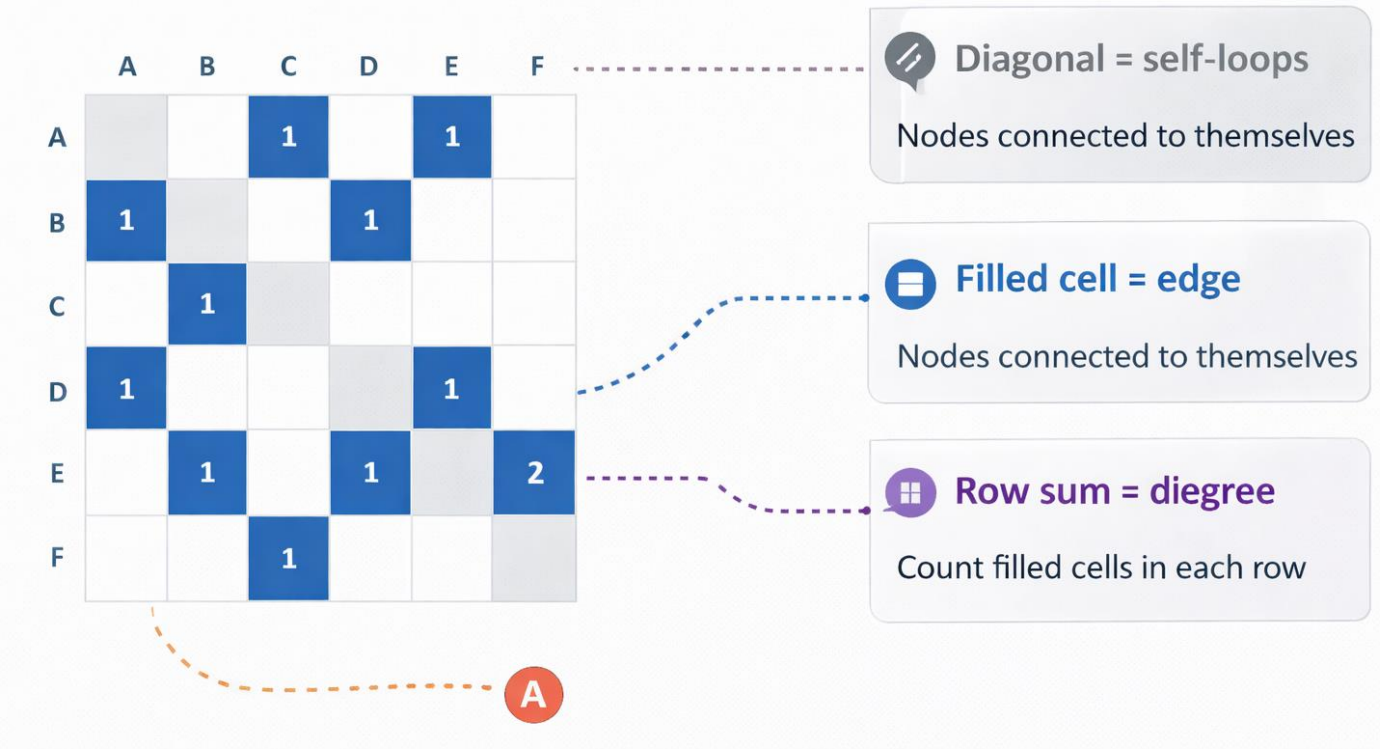
Node with zero connections — no edges at all.

Regular node

Average degree. Encodes data via size and color.

Anatomy of an Adjacency Matrix

Reading a matrix — rows are sources, columns are targets.



Diagonal = self-loops

Empty in simple graphs (no self-connections).

Filled cell = edge

Cell (A,B) filled → edge A→B exists.

Symmetric = undirected

If (A,B) = (B,A), the graph is undirected.

Asymmetric = directed

Different values at (A,B) vs (B,A) → digraph.

Row sum = degree

Count filled cells in a row to get node degree.

Key Metric: Graph Density

Density governs which representation to choose.

$$\text{Density} = 2|E| / (|V| \times (|V| - 1)) \quad \text{for undirected graphs}$$

0.00 – 0.05	0.05 – 0.30	0.30 – 0.70	0.70 – 1.00
Very sparse	Moderate	Dense	Near-complete

Rule of Thumb

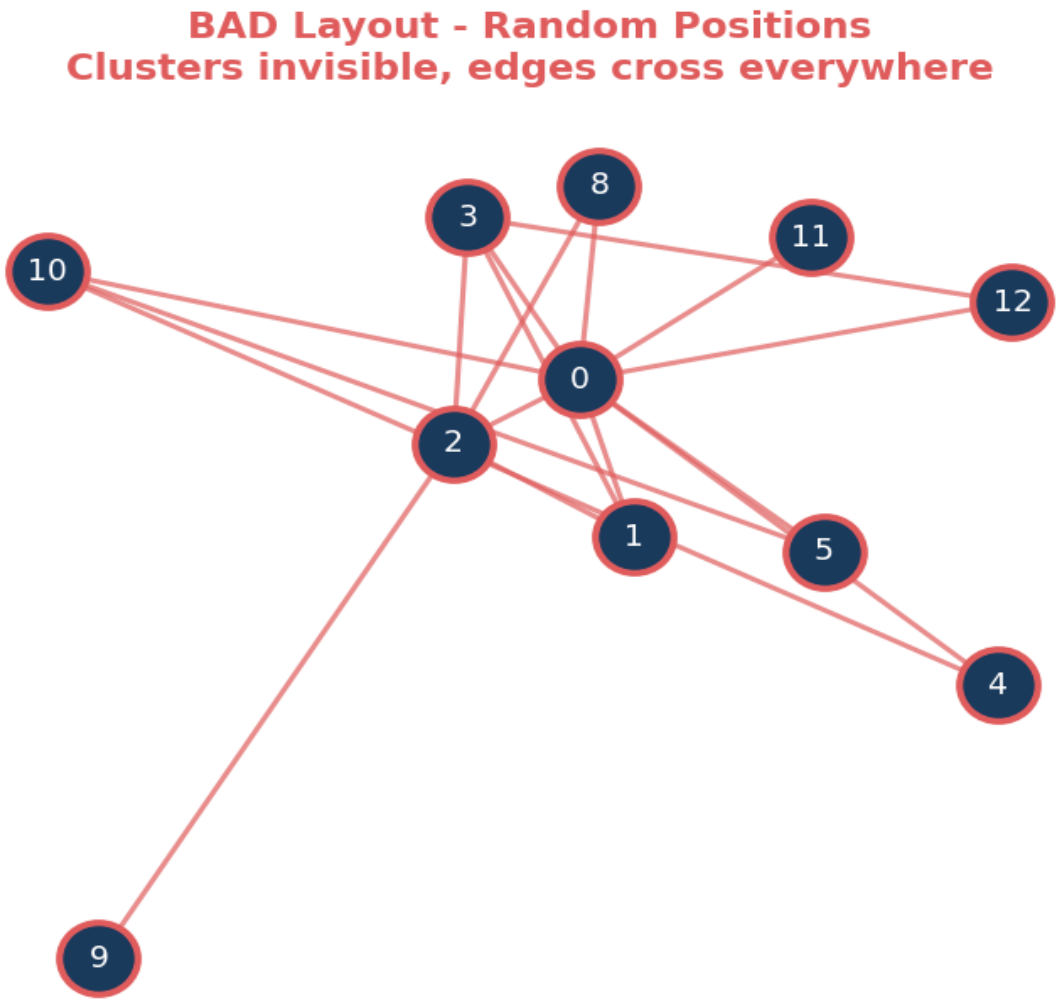
- NL if density < 0.3
- AM if density ≥ 0.3

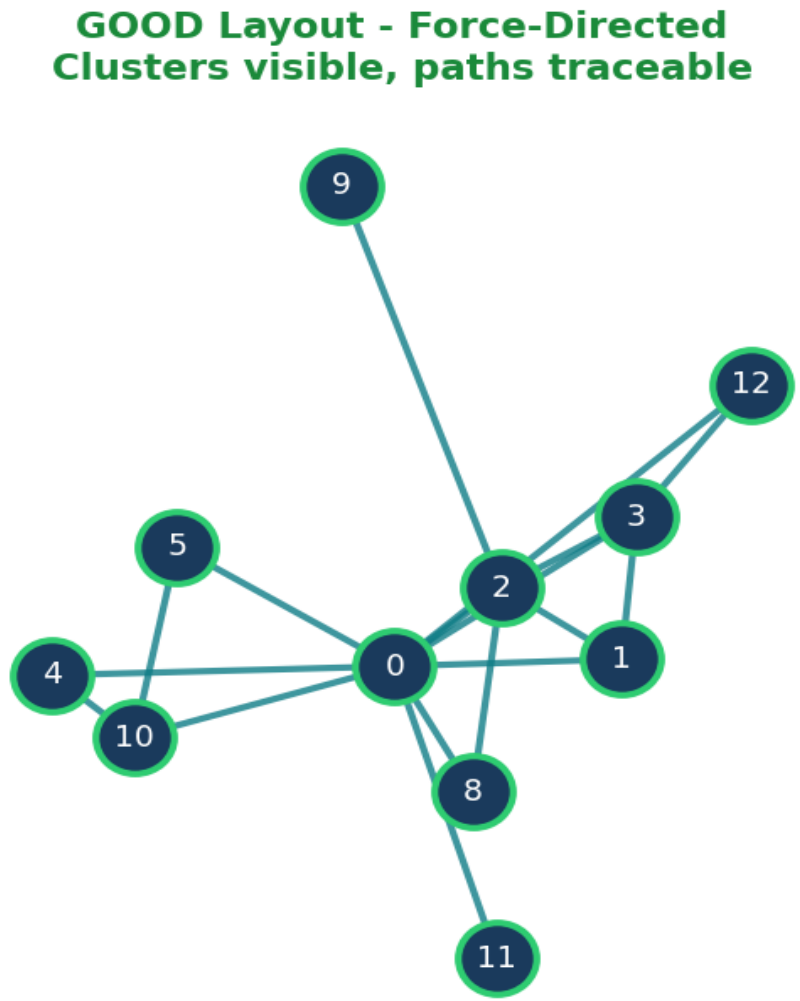
Density vs Node Count

Nodes	Sparse ($\|E\|=\|V\|$)	Dense ($\|E\|=\|V\|^{3/4}$)
10	0.22	1.00
50	0.04	0.98
100	0.02	0.99
200	0.01	0.99

Why Layout Matters: Same Data, Different Perception

Same 11-node graph — random positions vs. force-directed spring layout.





Edge Crossings

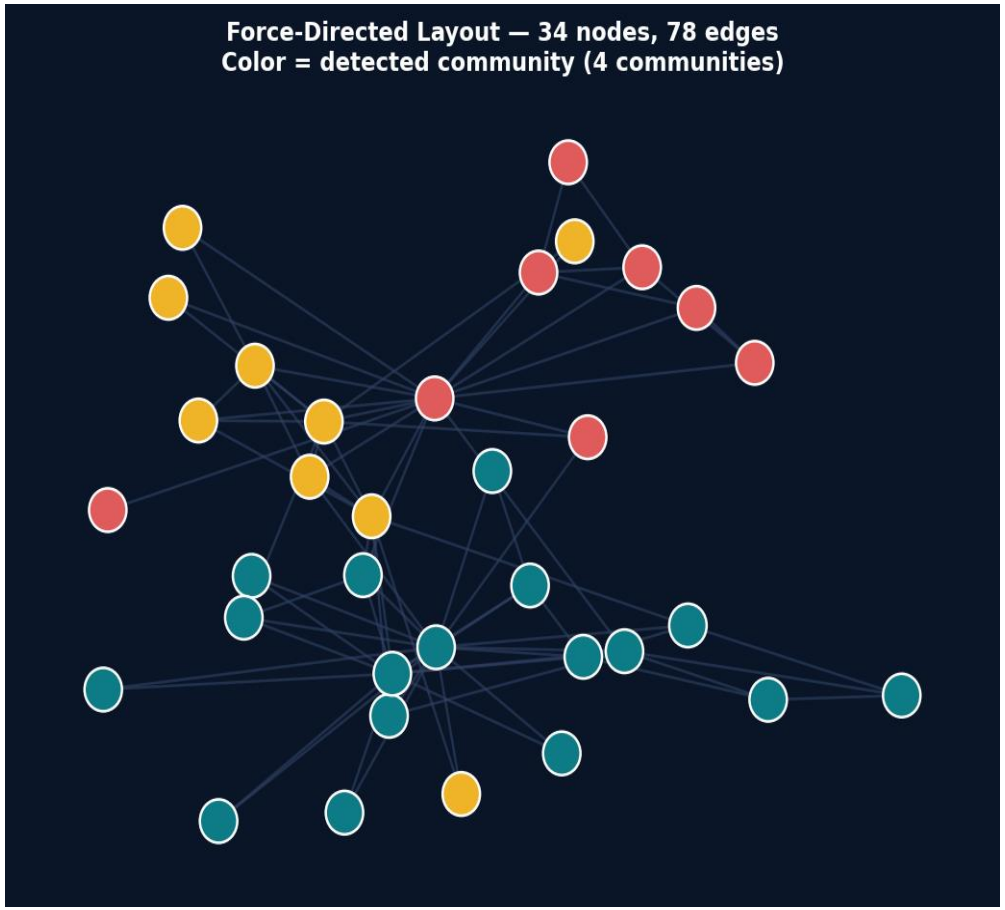
How visual design choices directly affect task performance and accuracy.

85%	Edge Crossings — increase in task time vs clean layout
75%	Node Overlaps — increase — labels hidden, grouping broken
65%	Unfamiliar Layout — increase — users struggle with non-standard arrangements
55%	Missing Labels — increase — identification tasks suffer most
45%	Excessive Color — increase — more than 7 hues overwhelms

% increase in task-completion time vs. clean layout — Adapted from Purchase et al. 2002

Introduction to Force-Directed Layouts

The most widely-used algorithm for general graph drawing.



Algorithms: Fruchterman-Reingold · ForceAtlas2 · D3-force

How It Works

Charged particles
Nodes repel each other like electric charges

Spring edges
Edges attract connected nodes like springs

Repulsion
Keeps nodes apart; prevents overlap

Equilibrium
Simulation ends when net force → 0

Topology
Spatial proximity reflects graph structure

Physics of Force-Directed: Hooke's Law & Coulomb's Law

Spring Forces — Hooke's Law

$F_{spring} = -k \cdot (d - L_0)$

k = spring stiffness constant
d = current edge length
L_0 = ideal (rest) length

Connected nodes attract like springs. Edge length converges to ideal L_0.

Repulsion — Coulomb's Law

$F_{rep} = k_r / d^2$

k_r = repulsion constant
d = distance between nodes

Every node pair repels like electric charges. Prevents overlaps.

Equilibrium: net force on all nodes → 0 — Layout stabilizes when attractive and repulsive forces balance.

Used in: D3.js forceLink · Gephi Yifan Hu · ForceAtlas2

Force-Directed: Advantages & Community Structure

Strengths

Reveals community structure automatically
No manual layout required
Topological proximity maps to spatial distance
Symmetric graphs → symmetric layouts
Widely supported: D3.js, Gephi, Cytoscape, yED

Limitations

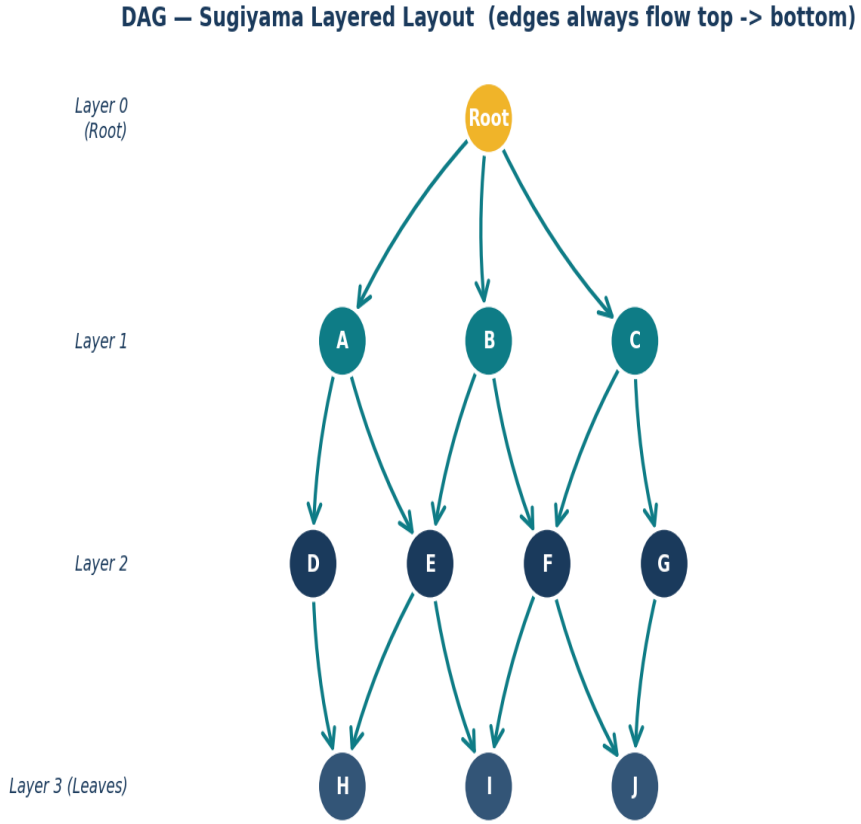
Slow: $O(n^2)$ per iteration
Non-deterministic results
Hairball > 200 nodes
Axis position has NO meaning

Best Use Cases

Social networks · Knowledge graphs · Citation networks · Exploratory analysis

Hierarchical Layouts: Visualizing Directed Acyclic Graphs (DAGs)

A DAG has direction but no cycles — perfect for hierarchies, pipelines & dependency trees.



Sugiyama Layered Layout — Key Concepts

Layered ranks

Nodes assigned to horizontal layers — edges always flow top-to-bottom.

Crossing minimization

Barycenter heuristic reorders nodes to reduce edge crossings.

Use cases

Git history, CI/CD pipelines, org charts, build systems.

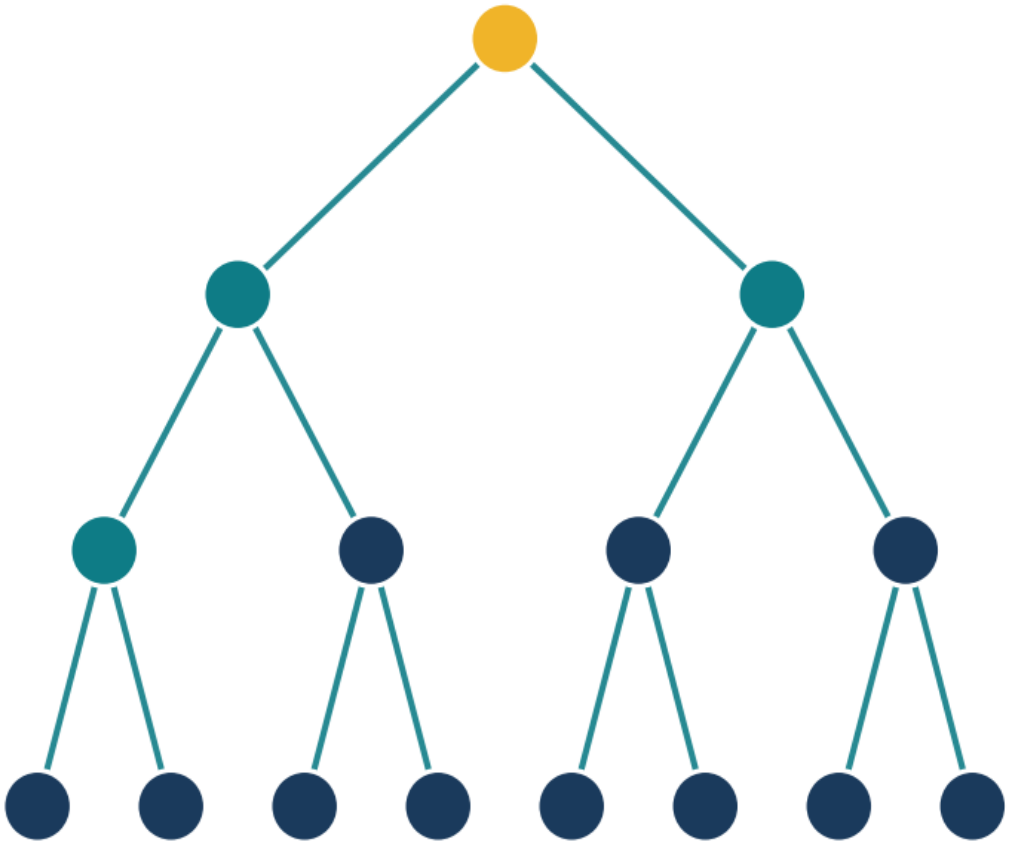
Libraries

Dagre.js · ELK · Graphviz dot

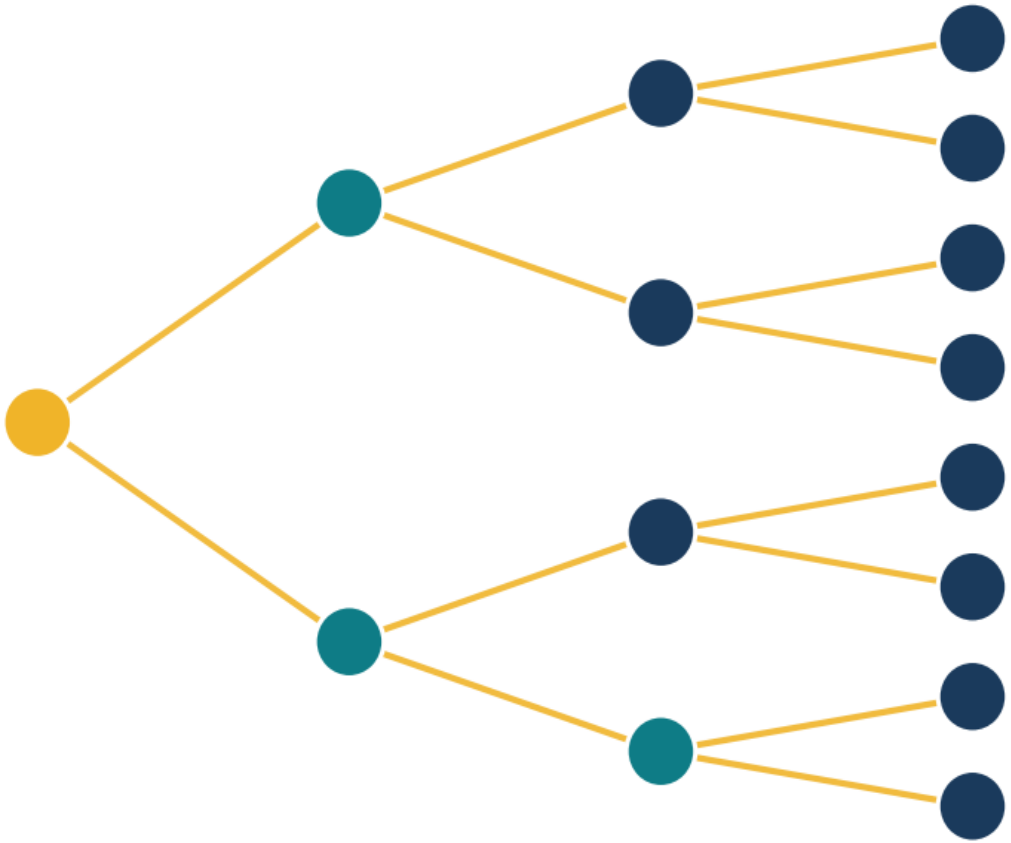
Top-Down vs. Left-to-Right Flows in Trees

Same binary tree (depth 3) drawn with two different layout orientations.

Top-Down (TB)
Org charts, family trees

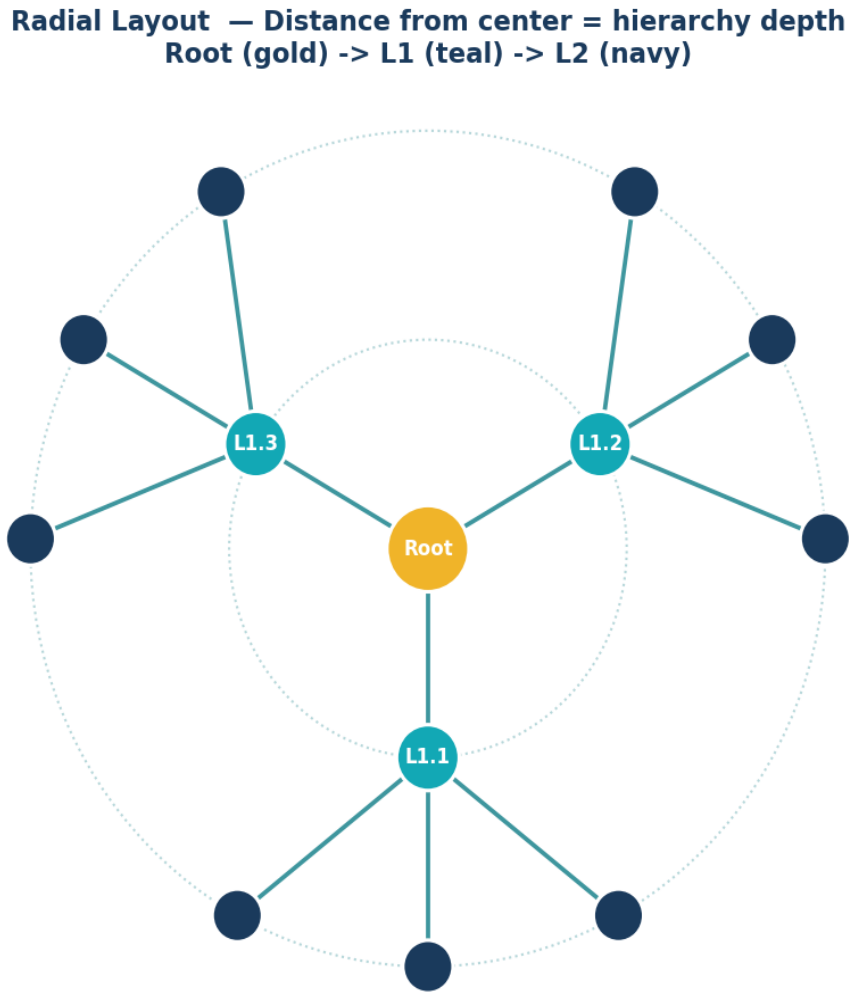


Left-Right (LR)
Pipelines, decision flows



Radial Layouts: Focusing on Centrality & Symmetry

Root node at center; distance from center = graph depth.



Center = root

Root placed at the center origin.

Depth = radius

Distance from center encodes graph depth.

Angular position

Subtree identity encoded angularly.

Concentric rings

Each ring shows one hierarchy level.

Ego-networks

Natural fit — spotlight on one node's neighborhood.

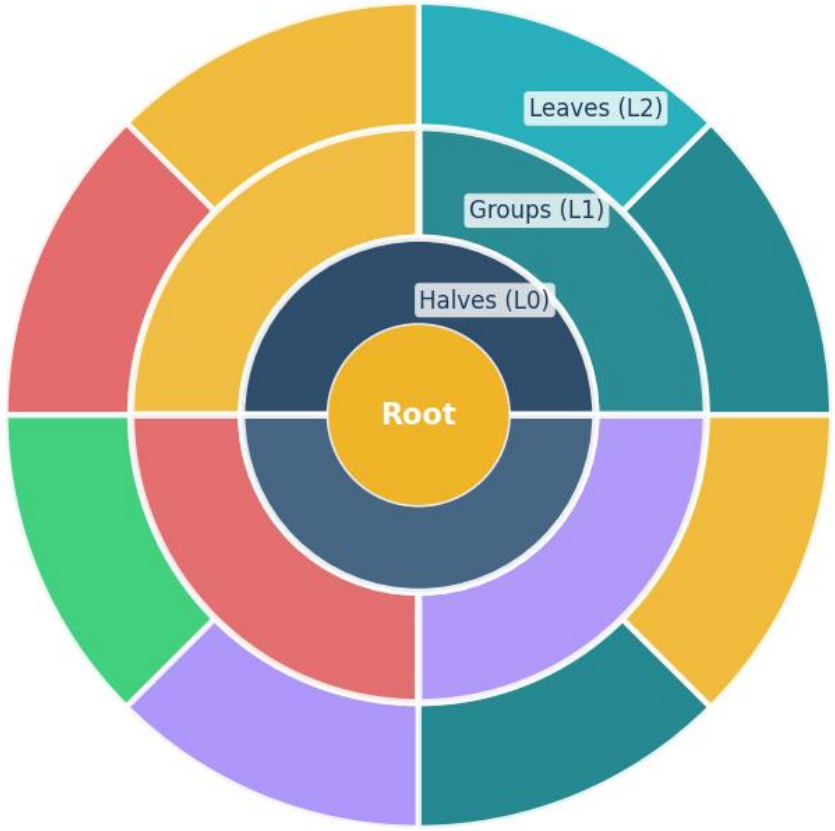
Tools

Gephi · yED · Cytoscape

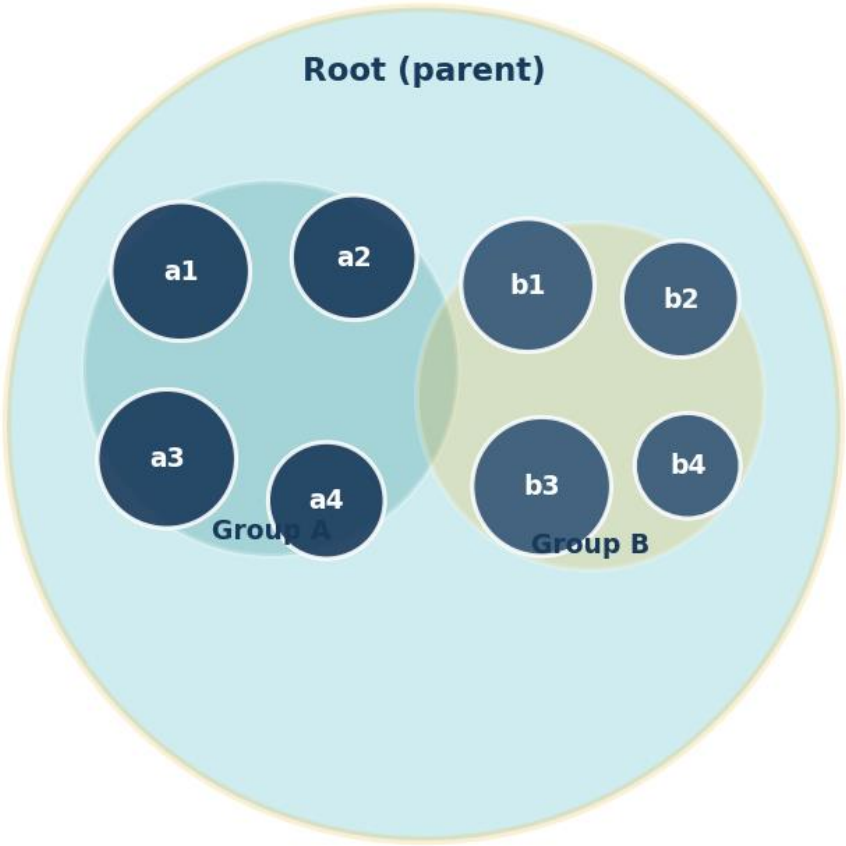
Radial Layouts for Large Hierarchies: Sunburst & Circle Packing

Both encode hierarchy using 2D space — ideal for showing large trees compactly.

Sunburst Chart
Inner ring = L0, outer rings = deeper levels (arc length = proportion)

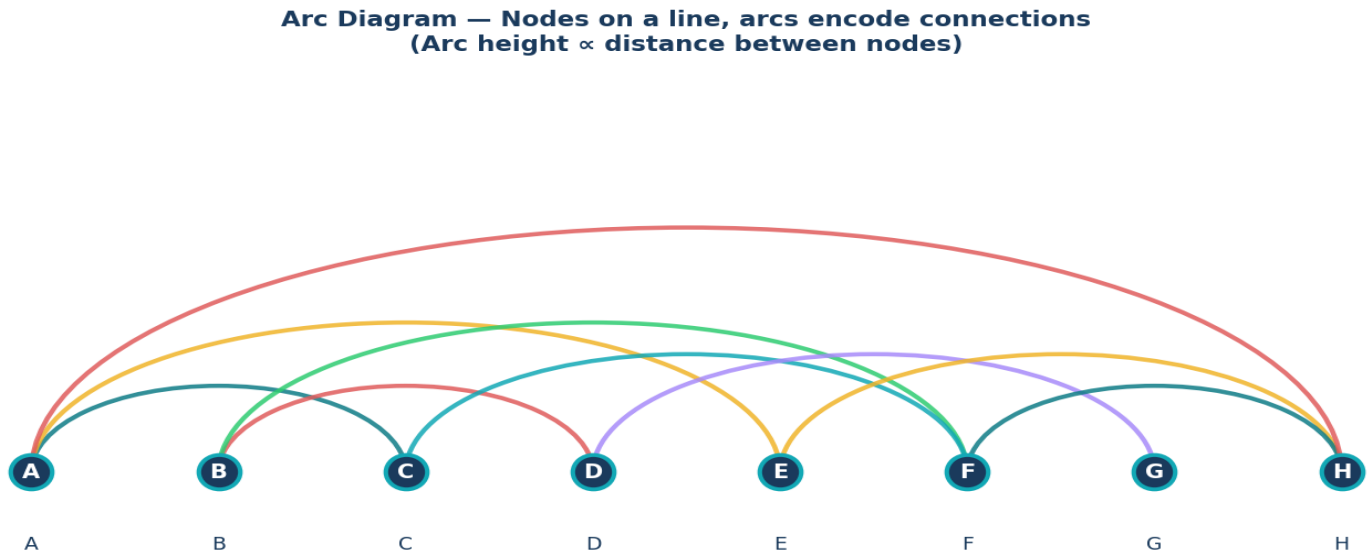


Circle Packing
Containment = parent-child (circle area = value)



Arc Diagrams: Linear Ordering & Longitudinal Paths

Nodes on a baseline; arcs encode connections above/below.



Proas

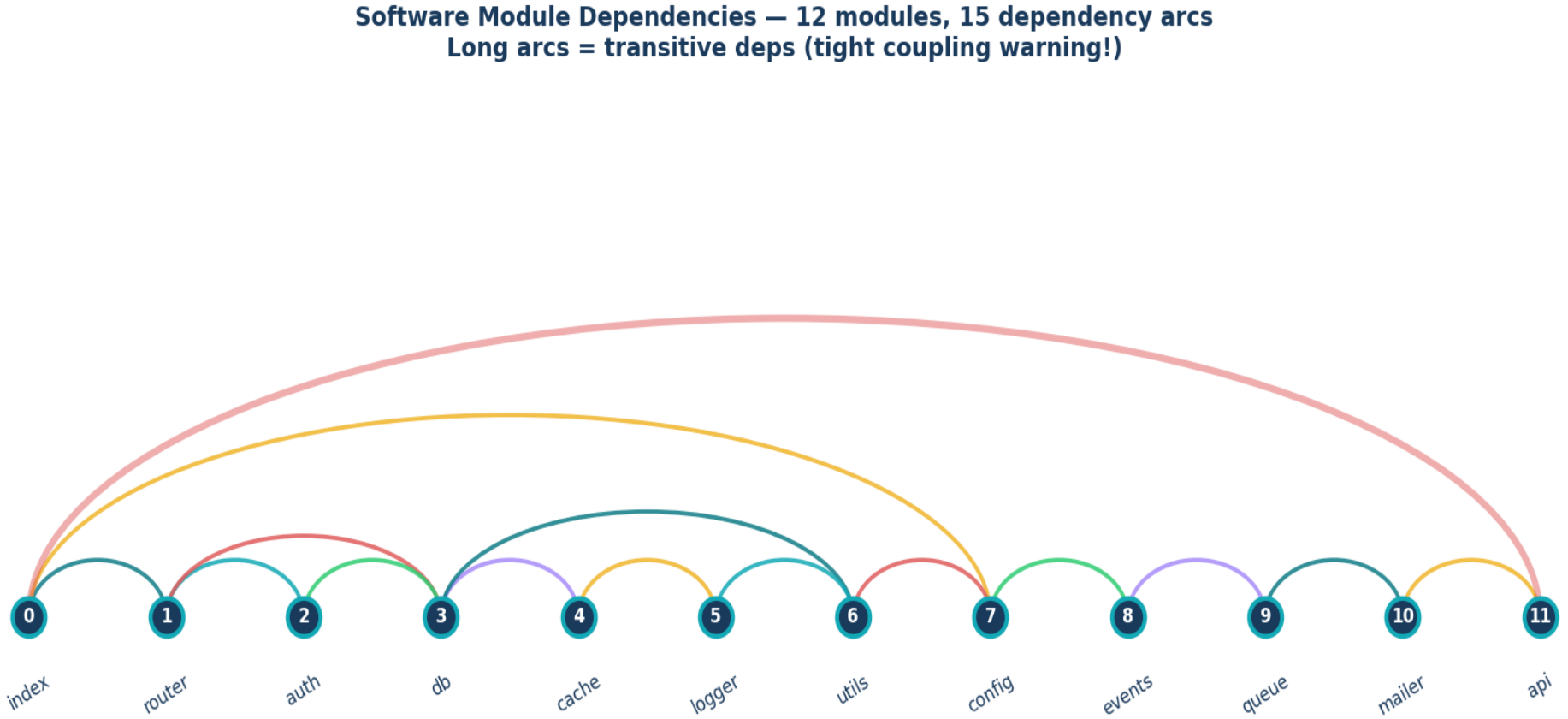
No node overlap — ever
Long-range patterns clearly visible
Works with meaningful ordering
Natural linear timeline fit

Cons

Arc crossings increase with density
Hard to encode node attributes
Requires meaningful ordering
Confusing when nodes > 50

Case Study: Arc Diagrams for Software Dependencies

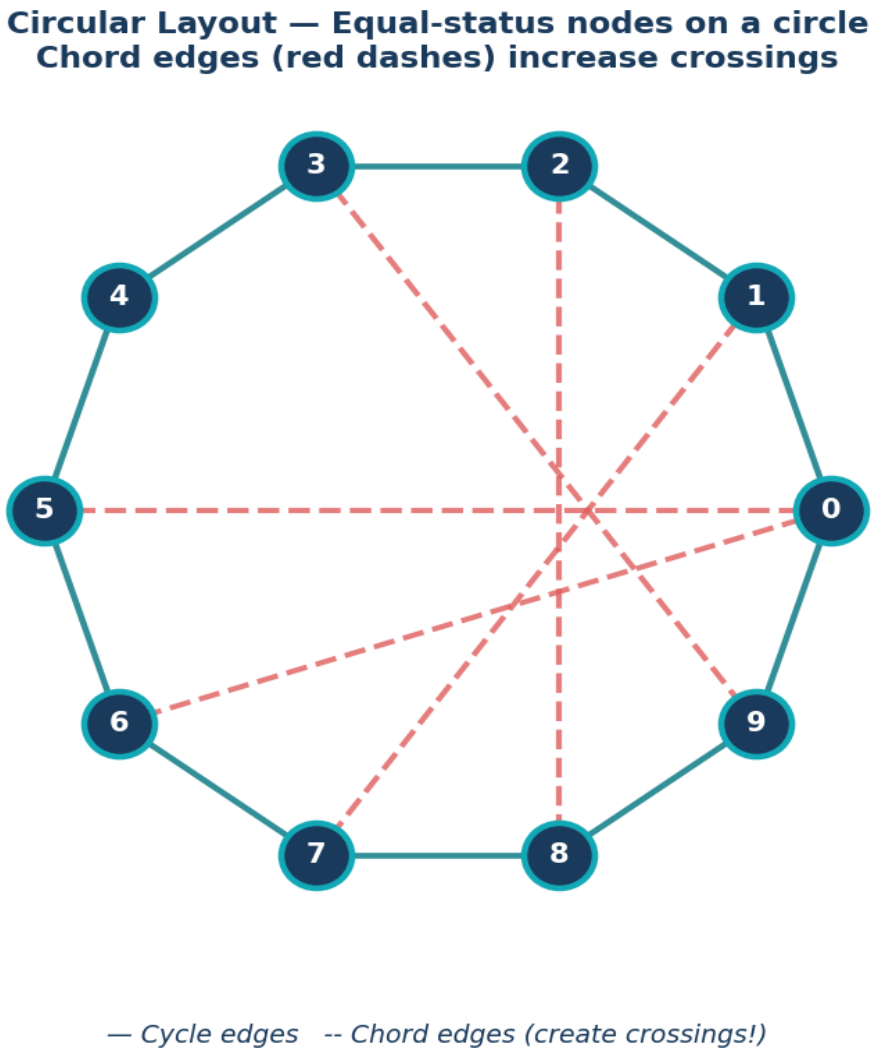
12 JavaScript modules in load-sequence order — 15 import dependencies visualized as arcs.



Long arcs reveal transitive dependencies skipping layers — a sign of tight coupling. Tool: Observable Plot · D3 arc · Gephi Arc Layout plugin

Circular Layouts: Simple but Prone to Crossover Clutter

Nodes on a circle — clean for small graphs, problematic when dense.



When Circular Works

Small number of nodes (< 20)
All pairwise comparisons equal
Chord diagrams (weighted edges)
No inherent hierarchy

Circular Pitfalls

Chord crossings grow as $O(n^2)$
No structural info in position
Axis position is arbitrary
Hides community structure

Summary Table of Layout Techniques

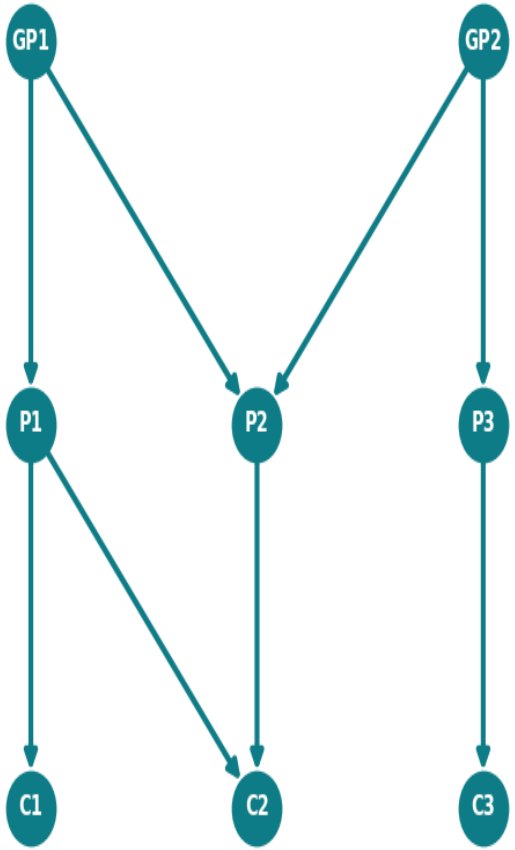
Choose the layout that matches your data structure and user task.

Layout	Best For	Scalability	Key Weakness
Force-Directed	General exploration, social nets	~1K nodes	Hairball; non-deterministic
Hierarchical (DAG)	Pipelines, org charts, CI/CD	High (with pruning)	Requires strict DAG
Radial	Ego-networks, centrality	~500 nodes	Overcrowding at outer rings
Arc Diagram	Ordered sequences, timelines	High (linear axis)	Crossing arcs when dense
Circular	Small equal-status sets	< 30 nodes	Reveals no structure
Adjacency Matrix	Dense, large graphs	10k+ nodes	Pathfinding difficult

Visual Exercise: Identifying the Best Layout

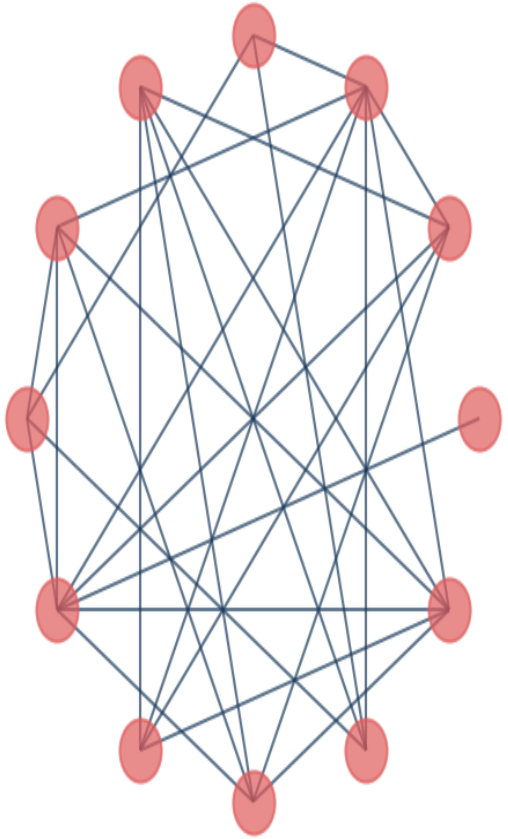
Discuss with a neighbour: which layout best fits each scenario, and why?

Family Tree
-> Hierarchical Layout

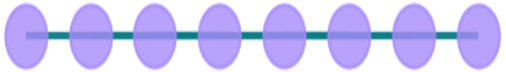


Social Network
-> Force-Directed

Protein Network
-> Adjacency Matrix



Sequential Data
-> Arc Diagram



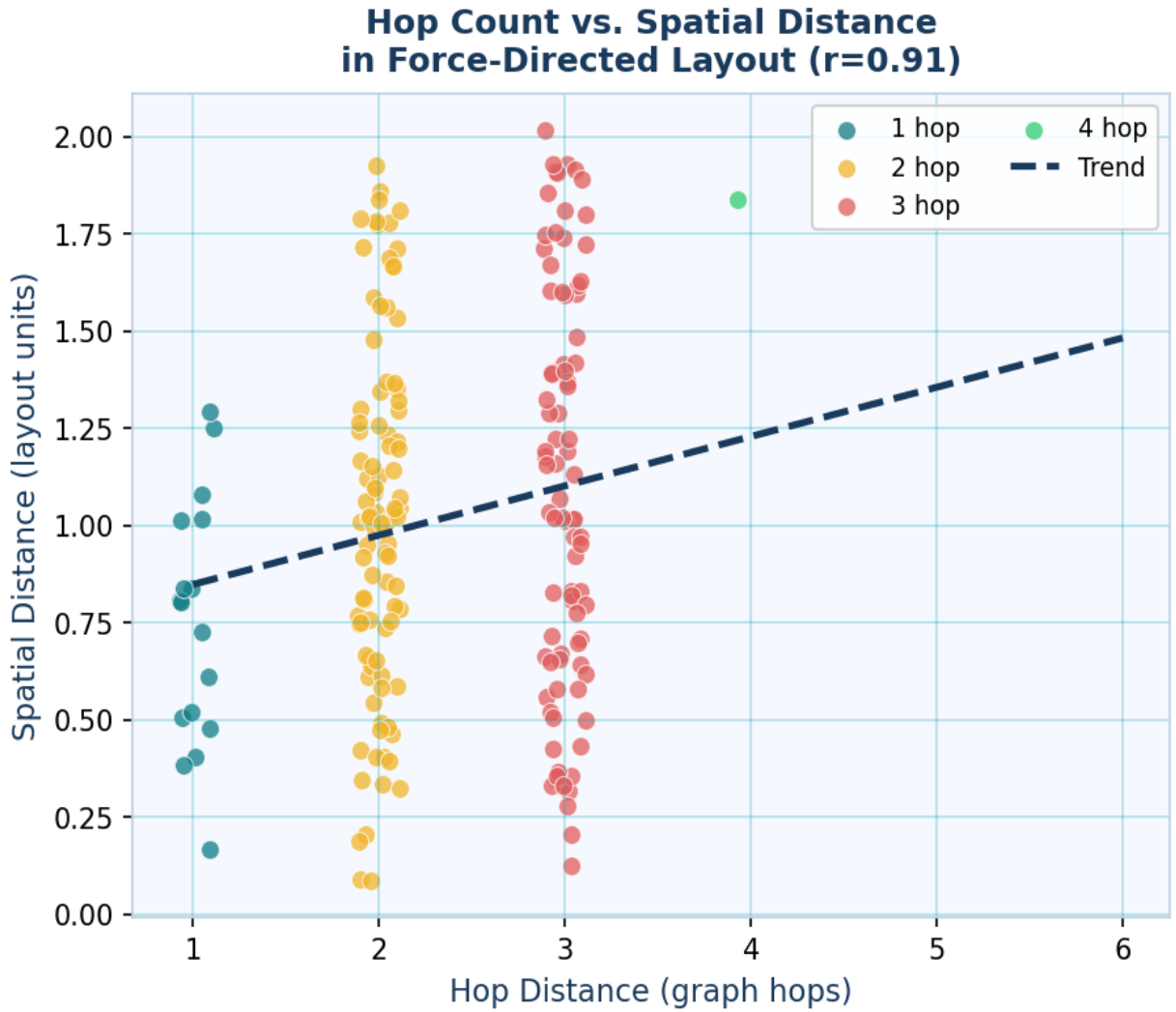
PART 3

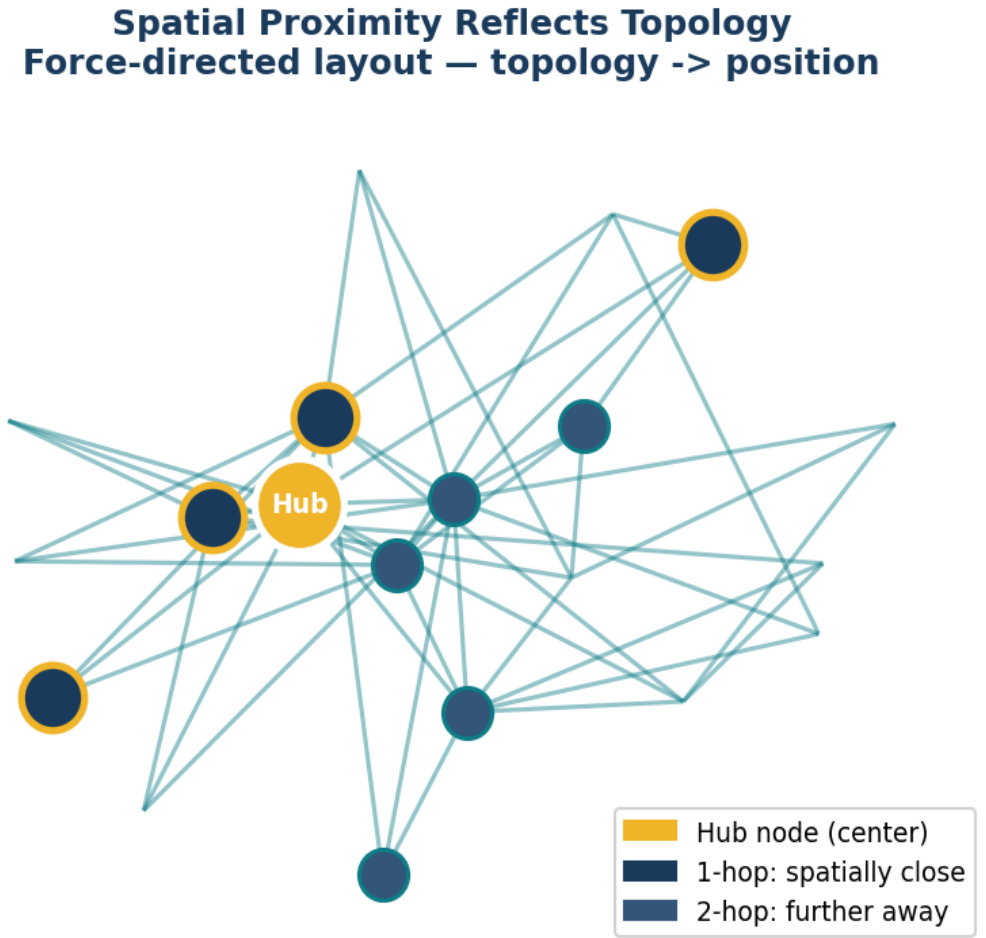
Node-Link Diagrams

Strengths, weaknesses, and when they work — and when they don't

When to Choose Node-Link: Small & Sparse Data

Rule: $|V| < 100$ AND Density < 0.3 $\Longrightarrow$ Node-Link is your friend

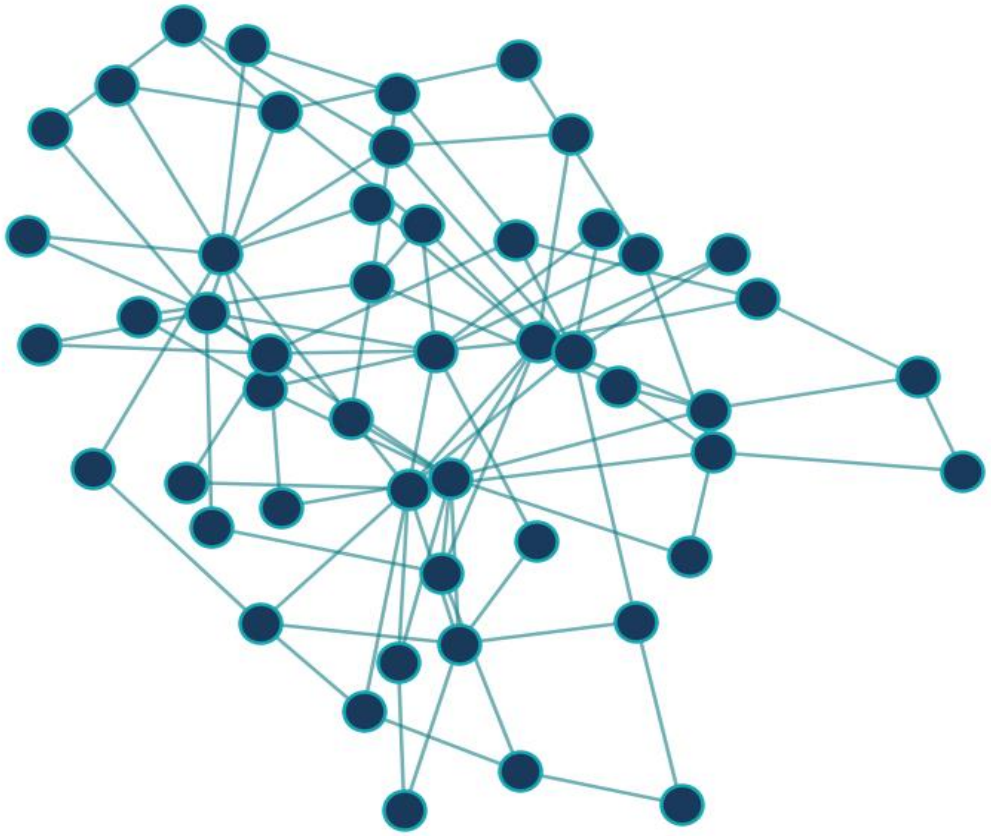




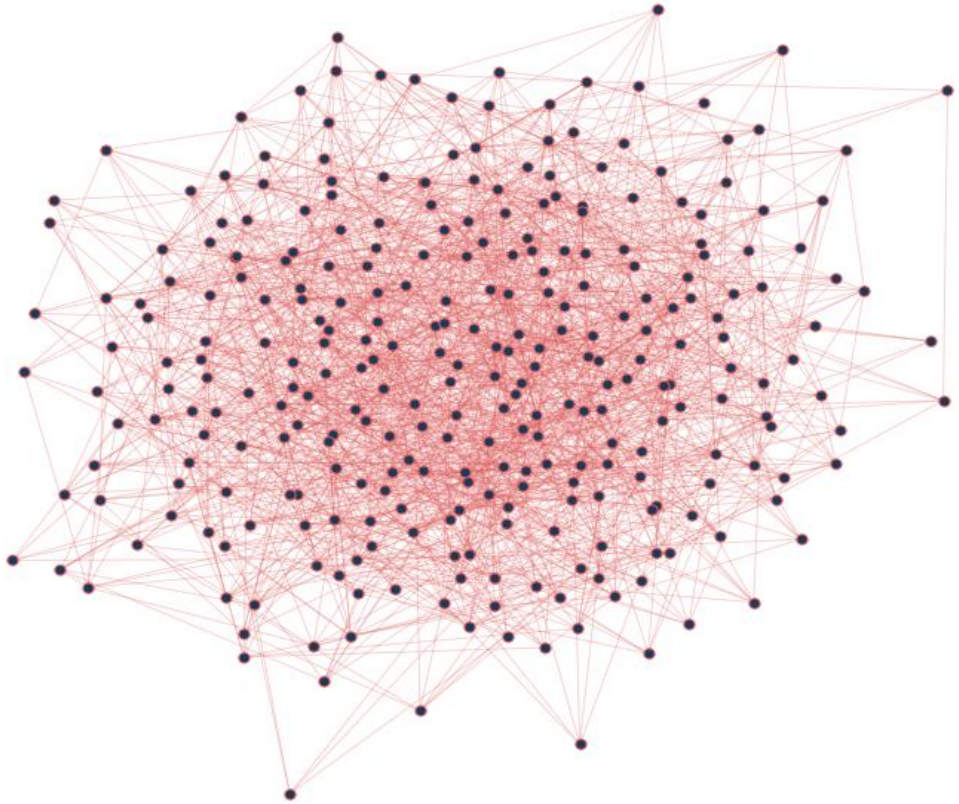
The Weakness: The "Hairball" Phenomenon

When node-link exceeds ~200 nodes it degrades into unreadable tangles.

GOOD: 50 nodes (BA model, sparse)
Readable — clusters & hubs visible



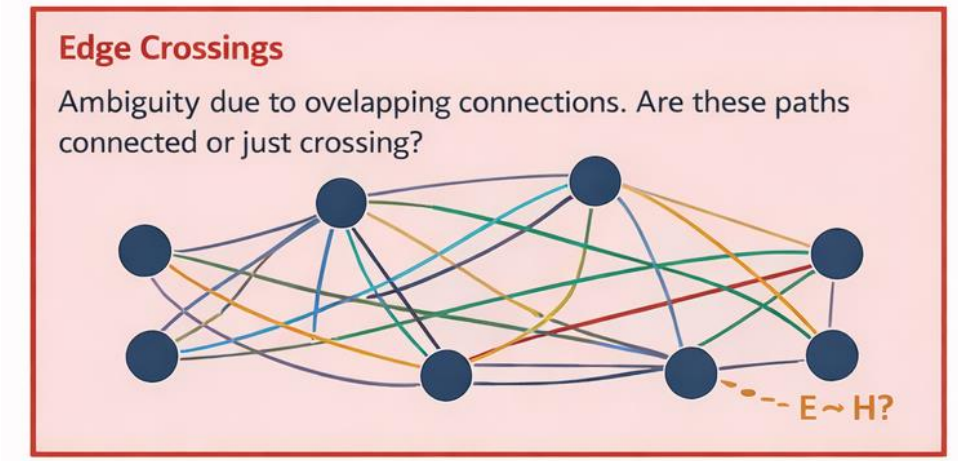
BAD: 300 nodes (ER random graph)
"Hairball" — structure unreadable!

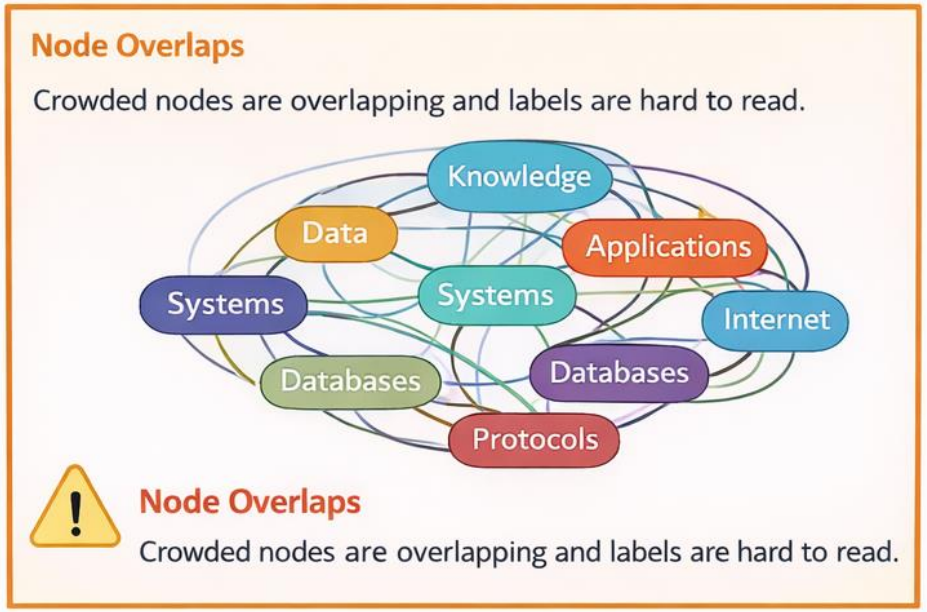


Visual Clutter: Edge Crossings & Node Overlaps

Purchase 1997 — error rate scales super-linearly with crossing count.

Crossings	Avg Error Rate (%)
0 crossings	5%
5 crossings	12%
10 crossings	24%
20 crossings	38%
40+ crossings	67%





Scalability Limits: Where Node-Link Starts to Fail

NL readability score degrades sharply after ~100 nodes.

Node Count	NL Readability	AM Readability
10	95	60
50	82	70
100	70	75
200	45	78
500	20	82
1,000	8	85
5,000	2	88

Key Insight

NL and AM have opposite scalability curves — they are complementary tools.

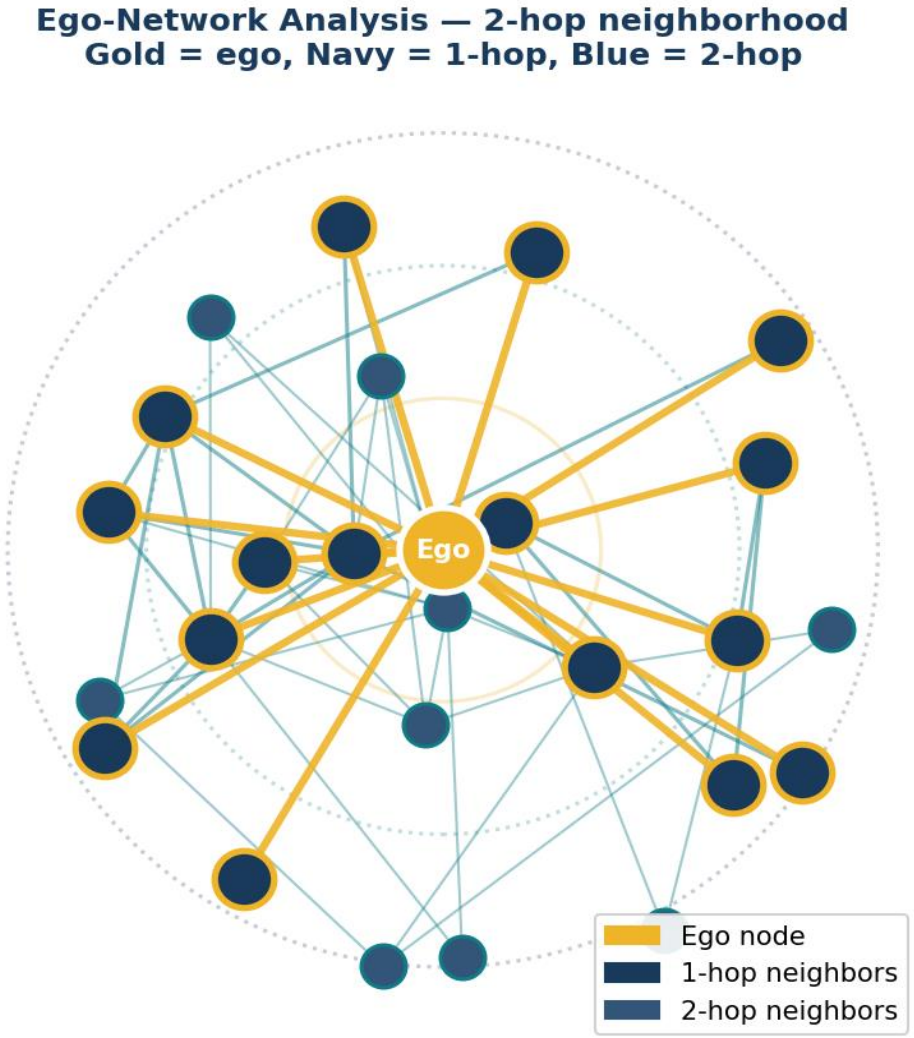
Below 100 nodes: NL dominates.

Above 100 nodes: AM is consistently more readable.

Hybrid approaches bridge the gap.

Use Case: Local Neighborhood Analysis (Ego-Networks)

Ego-network: focus on one node and its 1- or 2-hop neighbors. NL is ideal.



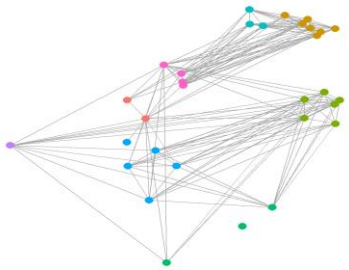
Applications

- Twitter follower analysis
- Citation network of a paper
- Friend-of-friend in LinkedIn
- Network security: blast radius
- Drug interaction neighbors
- Customer influence mapping

NL Best Practices: 6 Techniques to Reduce Visual Clutter

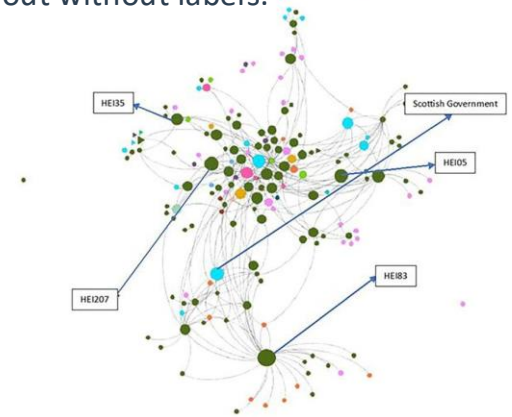
Node Color = Community

Categorical color encodes cluster membership. Use max 8 distinct hues.



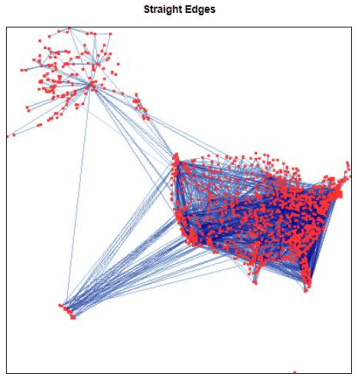
Node Size = Degree

Larger nodes signal higher degree. Hub nodes stand out without labels.



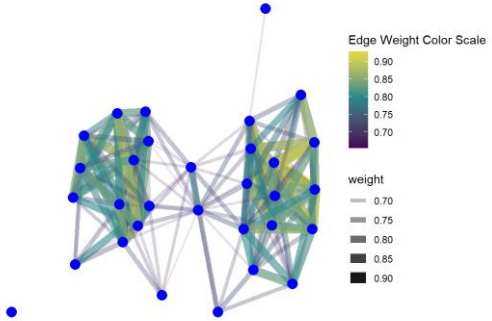
Edge Bundling

Group parallel edges into bundles. Reduces visual noise by 40–70% in dense graphs.



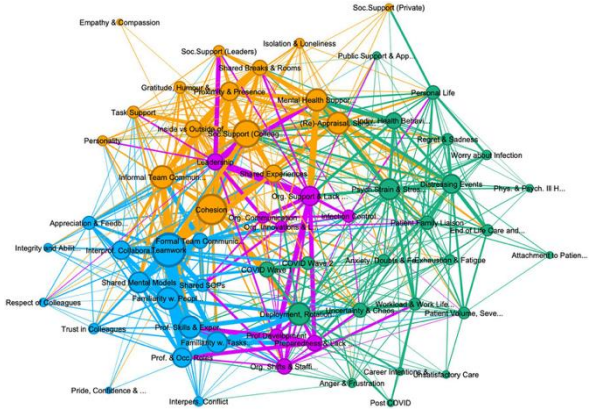
Edge Width = Weight

Thicker edges signal stronger relationships. Prune edges below threshold.



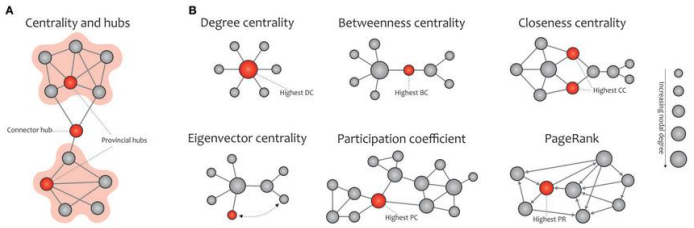
Semantic Zoom

Show labels at high zoom; cluster blobs at low zoom. Prevents overload.



Degree Filtering

Slider removes nodes below degree threshold. Exposes core structural skeleton.



PART 4

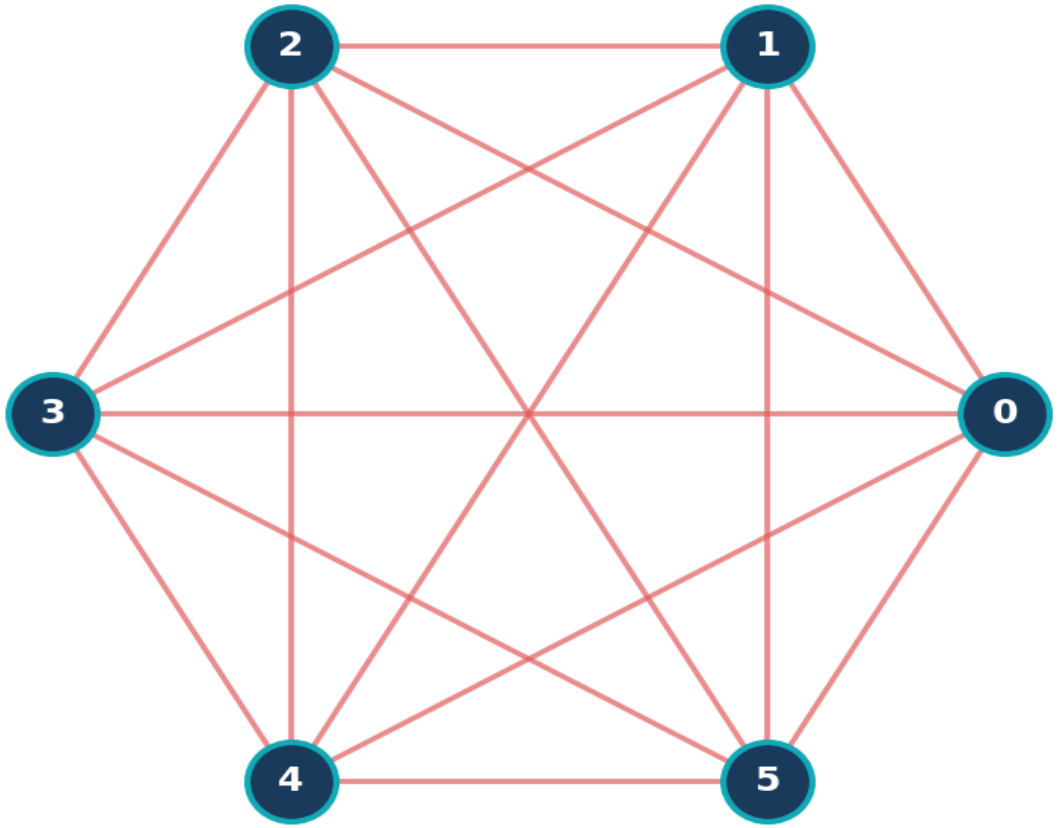
Adjacency Matrix Representations

Moving from links to grids

Why AM is Immune to Edge Crossings

In an AM every edge is a filled cell — there are NO lines to cross, EVER.

BAD: Node-Link (K_6 complete graph)
15 edges -> many crossings, hard to read



GOOD: Adjacency Matrix (K_6 complete graph)
15 edges -> zero crossings, instantly readable

	A	B	C	D	E	F
A		1	1	1	1	1
B	1		1	1	1	1
C	1	1		1	1	1
D	1	1	1		1	1
E	1	1	1	1		1
F	1	1	1	1	1	

AM Scalability: Visualizing Thousands of Nodes in a Single View

View Mode	NL Max Readable	AM Max Readable
With labels	50	100
No labels	150	2,000
Bundled	300	2,000
Clustered	800	10,000

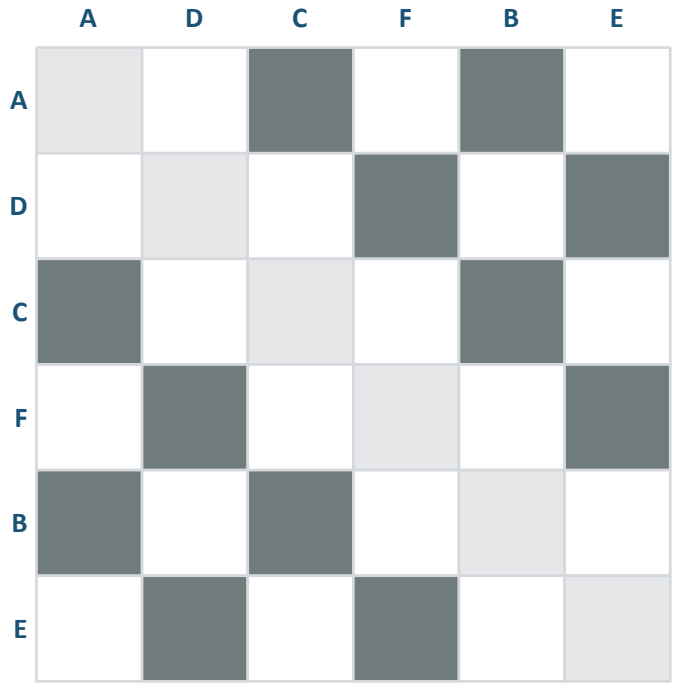
Why AM Scales Better

Each row/column is a fixed-width strip. Adding 1,000 nodes adds 1,000 strips — not 1,000 overlapping circles. Pixel-level rendering shows 10,000+ nodes in a single view.

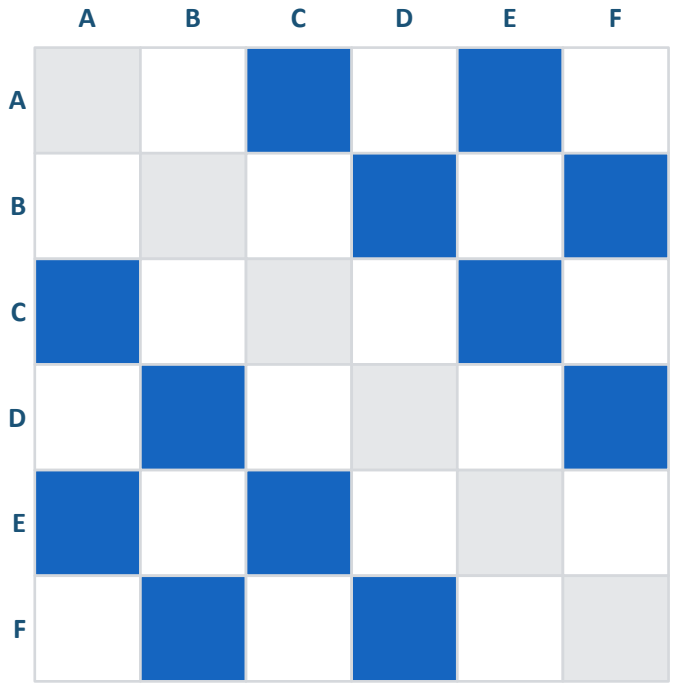
Detecting Clusters: The Importance of Matrix Reordering

Reordering rows/columns groups connected nodes into visible block-diagonal patterns.

Before Reordering



After Reordering (Seriation)



Clique = all cells in block filled

Block size = clique size k

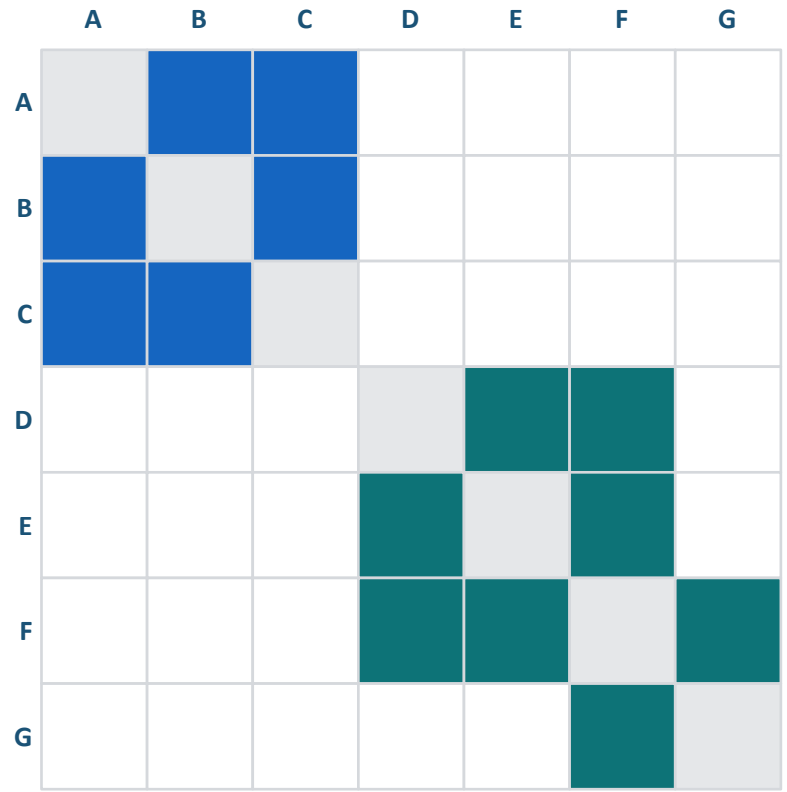
Multiple cliques = diagonal block pattern

Overlapping = shared rows/columns

Seriation algorithms: Reverse Cuthill-McKee · Spectral ordering

Patterns in Matrices: Finding Cliques (Solid Blocks)

A clique is a fully connected subgraph — it appears as a solid filled square along the diagonal.



Clique = all cells filled

Block size = clique size k

Multiple cliques = diagonal blocks

Overlapping = shared rows/cols

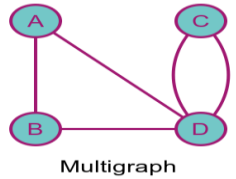
Applications: fraud rings · protein complexes · social cliques · research collaborations

Patterns in Matrices: Bridges & Off-Diagonal Connections

Off-diagonal cells reveal inter-cluster connections — bridge nodes linking separate communities.

Off-diagonal isolated cells

Weak ties — occasional connections between clusters.

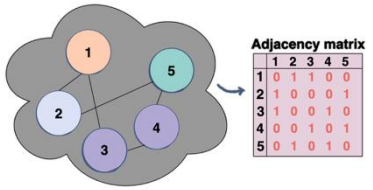


Multigraph

	A	B	C	D
A	0	1	0	1
B	1	0	0	1
C	0	0	0	2
D	1	1	2	0

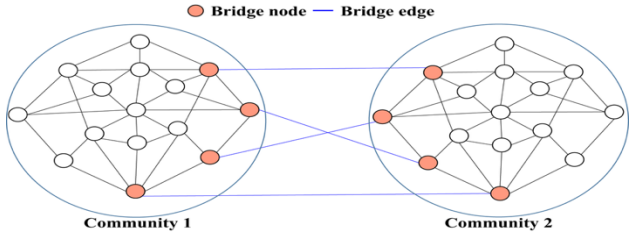
Off-diagonal bands

Structured coupling between two groups.



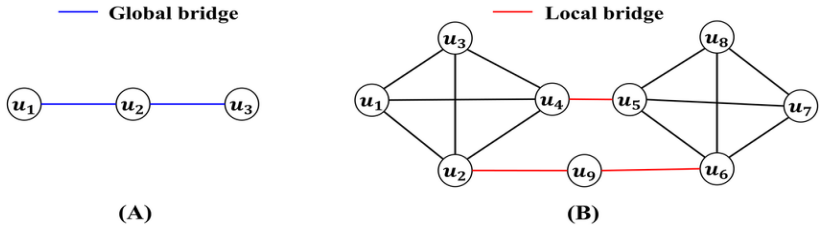
Bridge nodes

Appear in multiple block rows/columns — connectors.



Removing bridge

Disconnects the graph — critical vulnerability point.



Applications: supply chain vulnerabilities · social brokers · interdisciplinary collaboration networks

AM Weakness: Difficult Pathfinding — The "Mental Hop" Problem

Each hop requires scanning a full row AND column — 2 axis changes per step.

Mental Hop Procedure (A → D):

1. Step 1: Find row 'A' → filled at B, C
2. Step 2: Jump to col 'B' → find row B → filled at C, D
3. Step 3: D reached! 2 hops, 4 axis changes.

Why This Matters

Each hop requires the user to:
1. Scan an entire row
2. Switch to the column header
3. Repeat

For a 4-hop path → 8 axis changes.
Pathfinding in AM is a working memory workout.

Research: 3–4× longer than NL (Ghoniem et al. 2005)

Task	AM (sec)	NL (sec)
2-hop path	8	5
3-hop path	22	9
4-hop path	45	14
Cluster detect	12	35
Pattern ID	5	40

AM Weakness: High Learning Curve for Non-Experts

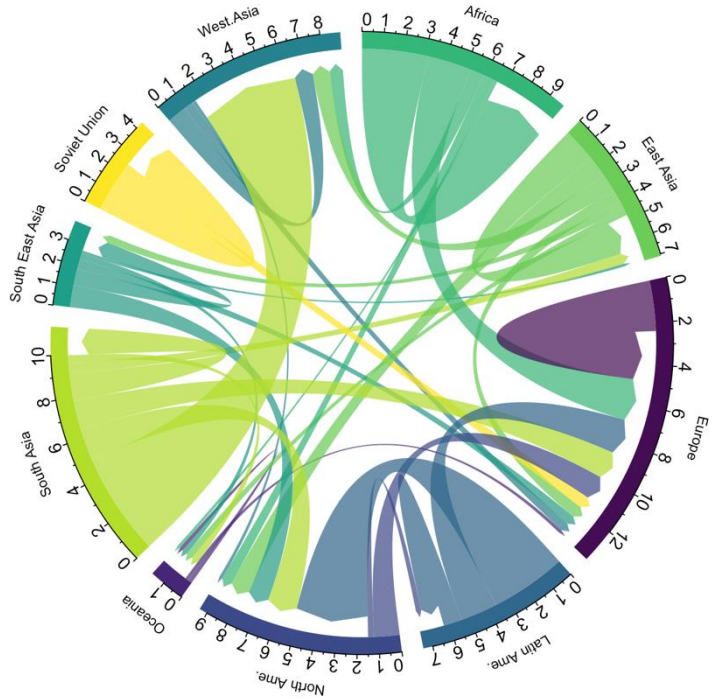
~60% of non-expert users cannot correctly read a basic adjacency matrix without training.

Expertise Level	Success Rate (%)
Expert (no training)	92%
After 5-min tutorial	85%
After 15-min tutorial	88%
Non-expert (no training)	38%
Non-expert (30-min training)	74%

Gibson et al. 2013

Design Mitigations

- Add row/column hover highlights
- Interactive tooltips on each cell
- Small NL overview alongside AM
- Use heatmap color not just 0/1
- Add clear legends & axis labels
- Consider MatLink hybrid approach



Optimizing the View: Sorting by Degree vs. Community

How rows and columns are ordered determines what patterns become visible.

Sort Strategy	Best For	Visual Effect	Algorithm	
By Degree (desc)	Finding hubs & outliers	Dense top-left → sparse bottom-right	Row sum sort	
By Community	Cluster detection	Block-diagonal structure	Spectral / RCM	
Alphabetical	Lookup by name	Predictable but structure-blind	None	
By Betweenness	Bridge identification	High-BC nodes appear centrally	Brandes O(VE)	
Random	Showing reordering value	No structure — comparison baseline	Math.random()	

NL vs. AM: Finding a Node vs. Finding a Pattern

Different representations excel at fundamentally different cognitive tasks.

Task	Node-Link	Adjacency Matrix
Find a specific node	Fast — direct visual search	Must scan all labels
Trace a 2-hop path	Follow edges visually	Mental hop required
Detect a cluster	Only with good layout	Solid diagonal block
Count node degree	Count edge lines	Count filled cells in row
Find global density	No clear visual encoding	Proportion of filled cells
Identify hub node	Largest / most central	Most filled row/column

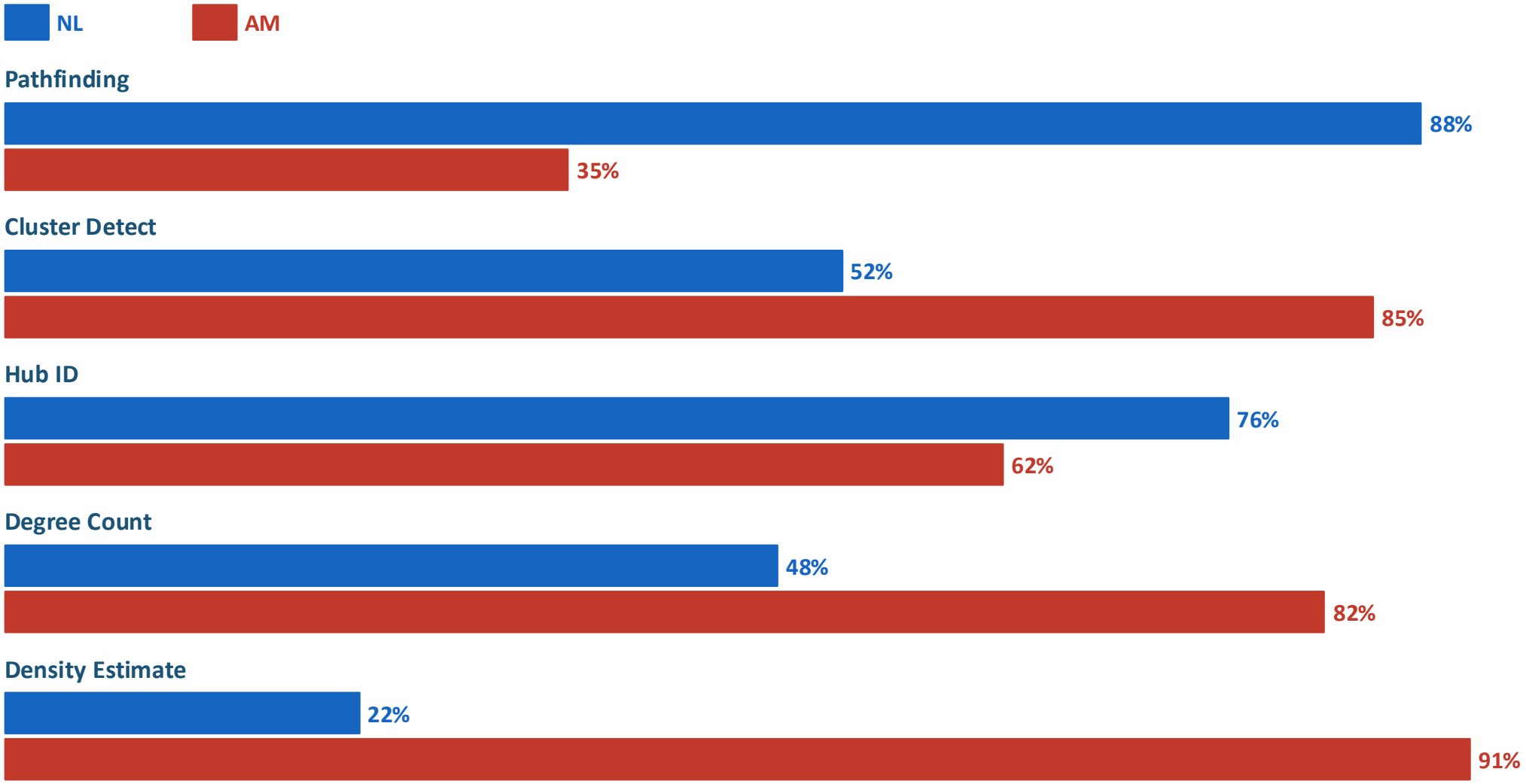
Head-to-Head: Small/Sparse vs. Large/Dense

A comprehensive comparison — choose the right tool for your data.

Property	Node-Link	Adjacency Matrix
Best graph size	< 100 nodes	100 – 10,000+ nodes
Best density	Sparse (< 0.3)	Dense (> 0.3)
Edge crossings	Common in dense graphs	Impossible — by design
Pathfinding	Intuitive	Mental hops required
Cluster detection	Moderate (layout dependent)	Excellent (block patterns)
Learning curve	Low — familiar metaphor	High — non-standard
Key tools	D3-force, Gephi, Cytoscape	D3, Reorder.js, Observable

Task Efficiency: Pathfinding (NL) vs. Global Structure (AM)

Task success rate — Ghoniem et al. 2005.



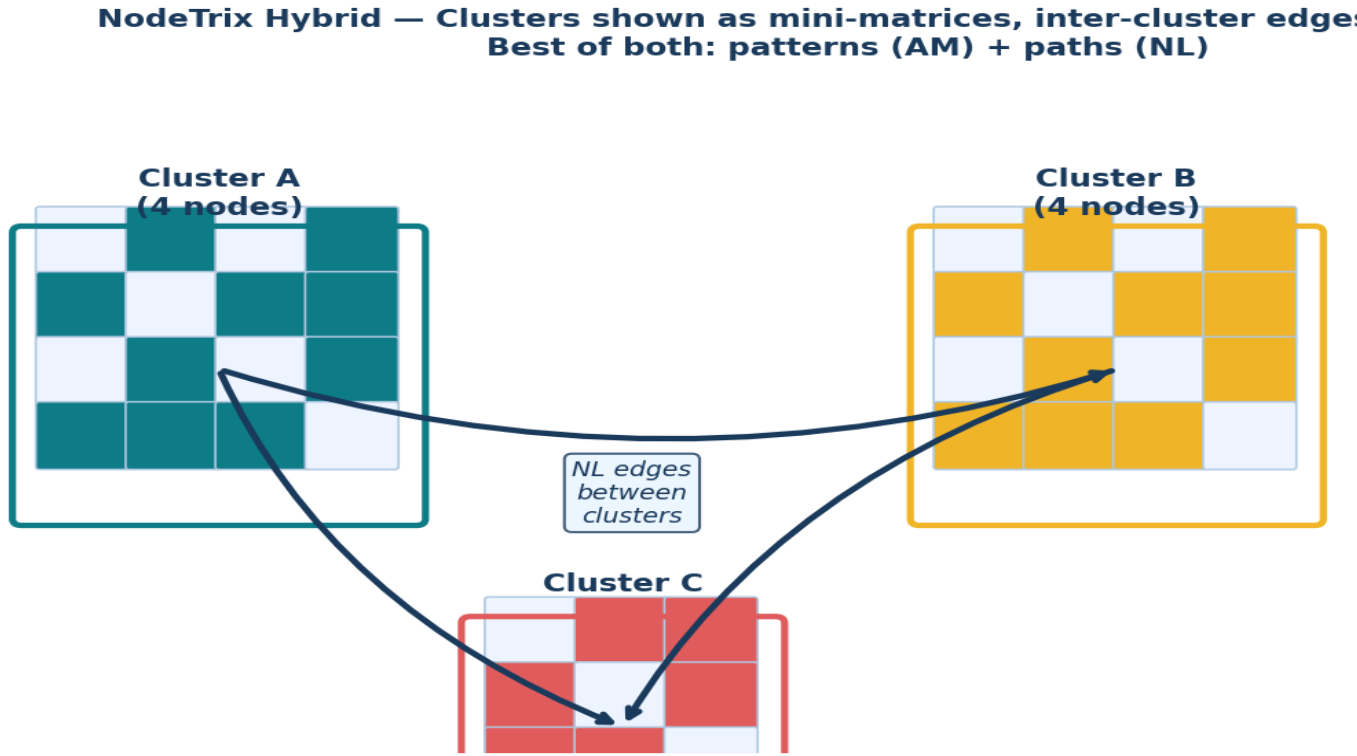
PART 5

Hybrid Approaches & Interactivity

Combining the best of both paradigms

Hybrid Approaches: MatLink & NodeTrix

Overlay node-link and matrix representations to preserve the strengths of both.



MatLink (Henry & Fekete 2006)

Matrix + arc diagram overlay. Arcs drawn outside matrix for key paths. Reduces mental hop burden.

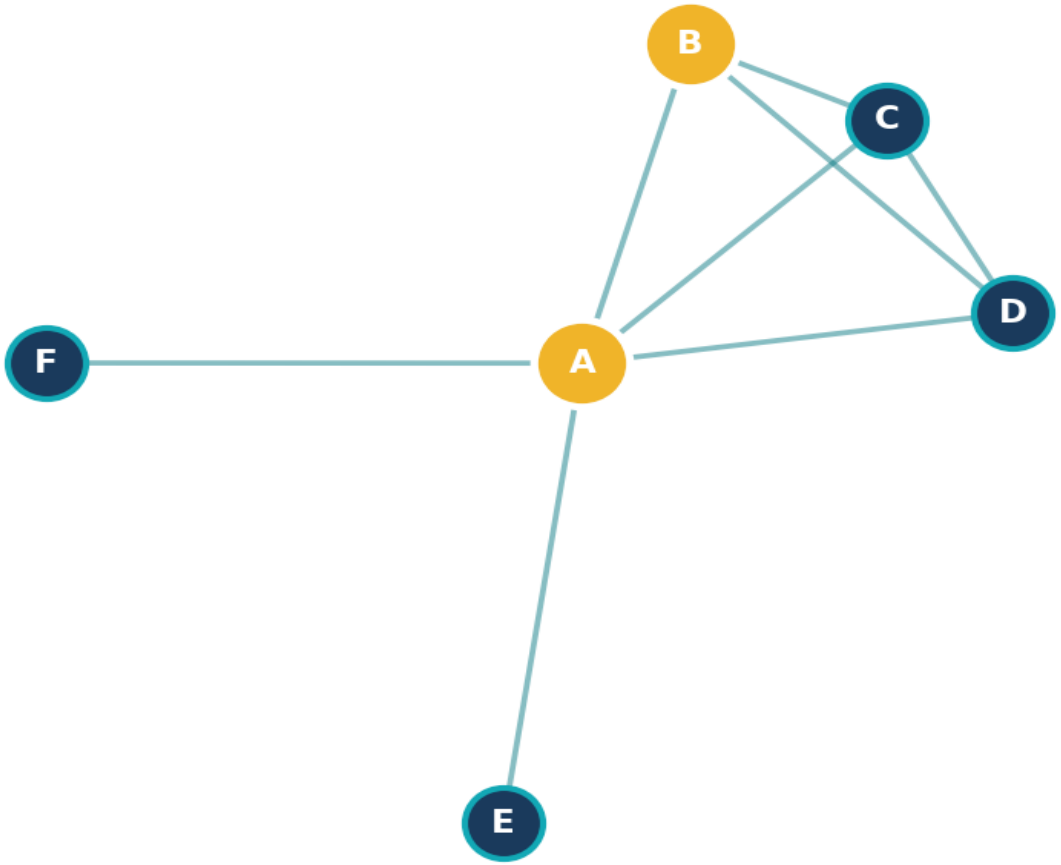
NodeTrix (Henry et al. 2007)

Dense clusters shown as matrices; sparse inter-cluster links as node-link edges. Best of both.

Multi-View Systems: Linked Brushing Between NL and AM

Selecting nodes in NL instantly highlights their rows/cols in AM — and vice versa.

NL View — Select nodes A & B (gold = selected)



AM View — A & B rows/cols highlighted (linked brushing from NL selection)

	A	B	C	D	E	F
A		1	1	1	1	1
B	1		1	1		
C	1	1		1		
D	1	1	1			
E	1					
F	1					

Zooming & Filtering: Interactive Graph Exploration

Six interaction techniques that make large graphs navigable.

Semantic Zoom

Low zoom = clusters; high zoom = individual nodes/labels. Prevents overload at every scale.

https://observablehq.com/@d3/force-directed-graph-component

Degree Filter

Slider removes nodes with degree < threshold. Exposes the core structural skeleton.

Focus + Context

Fisheye distortion magnifies area of interest while keeping global view visible.

https://bost.ocks.org/mike/fisheye/

Community Filter

Toggle entire communities on/off. Study one cluster in isolation.

https://www.polinode.com/

Time Slider

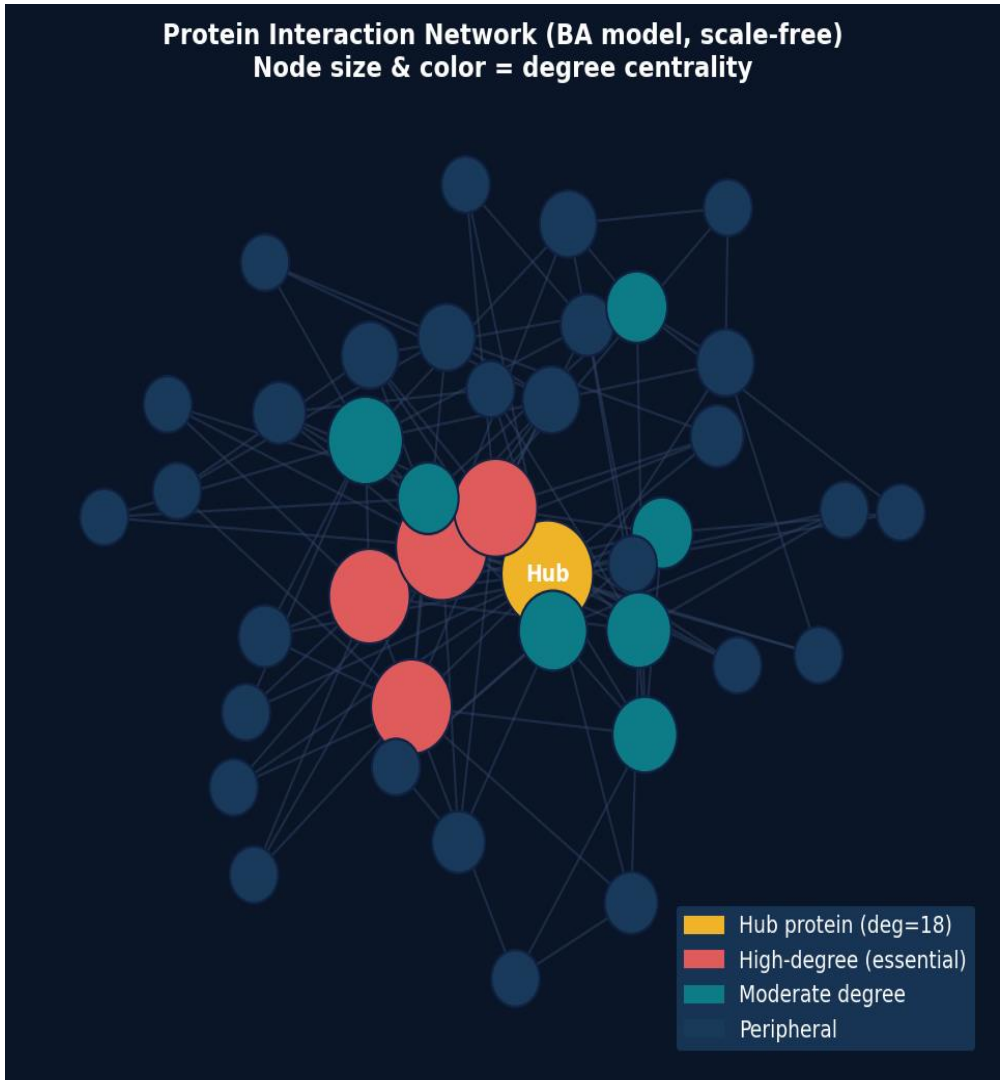
For temporal networks: animate edge appearance/removal over time.

Search + Highlight

Type node name → highlight node and neighbors; others fade to 20% opacity.

Case Study: Biological Networks & Gene Expression

20,000+ genes and millions of interactions in the human proteome.



Scale-free structure

Few hub proteins, many leaf nodes — power-law degree distribution.

Hub proteins

Essential for cell survival — disrupting them is often lethal.

AM + reordering

Reveals protein complexes as block-diagonal patterns.

WGCNA modules

Gene co-expression: edge = Pearson correlation; modules = pathways.

Tools

Cytoscape + STRING database · NetworkX · BioGRID

Tools for Implementation: D3.js, Gephi, Cytoscape

Use the right tool for your data, audience, and skill level.

D3.js

Web-based NL+AM+Force

Skill: Advanced (JS)

Maximum customization, animations, linked views. Observable integration.

Gephi

Desktop NL exploration

Skill: Beginner-friendly

Modularity detection, ForceAtlas2, rich plugin ecosystem.

https://gephi.org/

Cytoscape

Biological/molecular

Skill: Intermediate

Biological DB integration (STRING, BioGRID), SBGN notation.

Observable

Notebook NL+AM

Skill: Intermediate JS

Reactive notebooks. Great for teaching and prototyping.

NetworkX

Python analysis

Skill: Python users

Algorithm library. Pairs with Matplotlib or D3 for rendering.

Graphviz

Static hierarchical

Skill: Low

dot/neato engines. Best for DAGs, dependency trees, CI/CD diagrams.

Q & A + Final Review

"No single representation is best for all tasks — combine views for complex analyses."

Q: Which layout for a 500-node social network? → Force-directed + edge bundling, OR Adjacency Matrix if density > 0.3

Q: How do we detect cliques in large graphs? → Matrix reordering reveals solid diagonal blocks

Q: What is the main weakness of NL at scale? → The Hairball effect — edges tangle into unreadable messes

Q: Name one hybrid approach and its advantage. → MatLink / NodeTrix — preserves paths AND global patterns

Q: What density triggers the switch to AM? → Density > 0.3 OR node count > 100